ConvertInput-Free Vector Homomorphic Secret Sharing and Its Applications

Fan Yang
South China Normal University
yf202511@163.com

Fucai Luo
Zhejiang Gongshang University
lfucai@126.com

Xingfu Yan
South China Normal University
xfyan78@163.com

Haining Yang
Shandong University
hainingyang@sdu.edu.cn

Zheng Gong
South China Normal University
cis.gong@gmail.com

Wing W. Y. Ng
South China University of Technology
wingng@scut.edu.cn

Abstract

This work presents ConvertInput-Free Vector Homomorphic Secret Sharing (Vector-HSS), a novel HSS primitive based on the Decisional Composite Residuosity (DCR) assumption. Our construction enables efficient high-dimensional vector computations while avoiding the costly `ConvertInput` operation. As a unified framework, Vector-HSS can be used as a building block for Private Information Retrieval (PIR), Secure Multi-Party Computation (MPC), Privacy-Preserving Machine Learning (PPML), etc.

The key idea behind Vector-HSS is to introduce a vector-centric computation paradigm. Unlike traditional approaches, this design allows the server to perform natural and efficient vector computations without the `ConvertInput` operation while keeping client-side overhead comparable to that of state-of-the-art solutions. To further improve efficiency, we develop a batching mechanism based on the Chinese Remainder Theorem (CRT) that enables parallel computation across multiple vectors. Building on these techniques, we further design a suite of protocol modules to securely support Euclidean/cosine distance computation, comparison, and matrix-vector multiplication in practical applications. Experiments show that our scheme achieves 280× and 70× speedups over prior HSS schemes (EUROCRYPT 2021 and S&P 2026, respectively) for batched inner product evaluation. When applied to privacy-preserving image retrieval, our method achieves sub-second retrieval time, outperforming state-of-the-art solutions under similar security requirements.

1 Introduction

Homomorphic Secret Sharing (HSS), as a dual version of function secret sharing [11, 13], is an innovative cryptographic primitive that offers a distributed alternative to traditional homomorphic encryption [12, 16]. In an HSS framework, input data is split into secret shares and distributed across n servers ($n \geq 2$). Each server independently executes homomorphic computations on its local share to generate the corresponding result share. This "silent" computation capability makes HSS invaluable for both theoretical research and practical applications. Recently, it has been widely applied in vector processing scenarios, including accelerating Private Information Retrieval (PIR) [21,27,43], optimizing Secure Multiparty Computation (MPC) [6,10,19,24,34], and privacy-preserving machine learning (PPML) [2,5,28,31].

The secure computation implementation of existing HSS schemes typically relies on Restricted Multiplication Straight-line (RMS) programs [1, 9, 12, 22, 23, 30, 38]. This model imposes structural constraints on multiplication operations, and this constraint is precisely one of the key reasons for the low execution efficiency of HSS in high-dimensional vector computation scenarios. Specifically, RMS only allows multiplication between input values and previously computed memory values, requiring fresh inputs to first be converted into memory values via a dedicated procedure called `ConvertInput` [38, 40]. While a single convert operation incurs relatively low overhead, it must be invoked repeatedly in vector computations, eventually becoming a dominant performance bottleneck. In addition, to ensure the correctness of the distributed discrete logarithm (**DDLog**) computation involved in `ConvertInput` under modular arithmetic, existing schemes (e.g., [38]) typically need to introduce key decomposition technique to prevent modular overflow. While this approach guarantees correctness, it inevitably increases the number of `ConvertInput` calls, leading to substantial computational overhead.

Therefore, in high-dimensional settings, the computational cost of `ConvertInput` grows with the input dimensionality and is further exacerbated by key decomposition technique. As a result, the overall computation cost scales linearly, or even super-linearly, with the input size. This makes existing HSS schemes inherently inefficient for large-scale vector and matrix computations. Although several optimizations [1, 30] have been proposed to reduce either the cost or the invocation frequency of `ConvertInput`, their key computations still fundamentally depend on this operation. Consequently, the computational cost continues to grow with the input dimensionality, rendering these approaches unsuitable for high-

Table 1: Computational Complexity Comparison with Existing HSS schemes

Scheme	Assumption	Error Rate	Computational Complexity	
			Data Preparation (Client)	ConvertInput (Server)
[12]	DDH	$1/\text{poly}$	$O(2d\log^3 p)$	$O(d(\ell+1)\log^3 p)$
[38], [1]	DCR	negl	$O(2d\log^3 N)$	$O(d(\ell+1)\log^3 N)$
[30]	DCR	negl	$O(d\log^3 N)$	$O(d\log^3 N)$
Vector-HSS	DCR	negl	$O(d\log^3 N)$	None

Notations. d denotes the vector dimension, ℓ denotes the number of digits after key decomposition, p denotes a large prime, and N denotes the product of two large primes.

dimensional settings. This naturally leads to a fundamental question: *Can we design a new HSS scheme that eliminates the reliance on* `ConvertInput` *while still efficiently supporting high-dimensional vector computations?*

1.1 Our Contributions

In this work, we answer this question affirmatively and propose a Vector-HSS scheme based on the DCR assumption, which is ConvertInput-free for efficient and secure high-dimensional vector computation. Building on this, we design a standardized set of protocol modules that support secure vector computation in settings such as PIR, MPC, and PPML. As representative applications, we validate the efficiency and effectiveness of our proposed Vector-HSS by instantiating a privacy-preserving image retrieval scheme and a secure two-party inference framework built upon our protocol modules.

ConvertInput-Free Vector-HSS Scheme. We propose Vector-HSS, a ConvertInput-free homomorphic secret sharing scheme specifically designed for efficient high-dimensional vector computation. Different from existing HSS schemes that implement inner products via repeated ConvertInput operations under the Restricted Multiplication Straight-line (RMS) paradigm, our scheme adopts a vector-centric computation structure that eliminates the need for per-element conversion and enables more direct evaluation over vector shares. This structural design incurs no extra overhead during the client-side preparation phase and reduces the computational burden on the server side. Table 1 compares the computational complexity of representative existing HSS schemes with our scheme in high-dimensional vector computation scenarios. Moreover, Vector-HSS further improves the efficiency of high-dimensional vector computation by reducing the number of exponentiation operations.

To improve the practicality of Vector-HSS in real-world scenarios, we introduce a batching mechanism based on the Chinese Remainder Theorem (CRT), which allows the inner product computation between a fixed vector $\mathbf{x}$ and multiple vectors to be packaged into a single protocol execution. In addition, we design a set of protocol modules for the Vector-HSS setting, including secure Euclidean/cosine distance calculation, secure comparison, and secure matrix-vector multiplication protocols. These modules enable flexible deployment of Vector-HSS in high-dimensional vector computation scenarios such as PIR and PPML. Formal security analysis confirms that the proposed scheme achieves provable security, and experimental results show that Vector-HSS provides an efficient solution for large-scale secure vector computation.

Vector-HSS Application and Performance Verification. To demonstrate the application value of Vector-HSS, we construct and instantiate a representative application scenario for PIR, a privacy-preserving image retrieval algorithm based on the proposed protocol modules. Furthermore, we construct a secure outsourced two-party inference framework for PPML, thereby validating the practicality and extensibility of Vector-HSS.

1. Building on the above protocol modules, we propose a privacy-preserving image retrieval algorithm based on Vector-HSS. It employs a hierarchical clustering tree to construct a layered index structure and leverages batched Vector-HSS for efficient secure vector computation, enabling efficient layer-by-layer filtering and retrieval in the encrypted domain. Experimental results demonstrate that our Vector-HSS-based image retrieval algorithm achieves sub-second retrieval time. It outperforms existing methods based on secure K-Nearest Neighbors (KNN) [41] and Fully Homomorphic Encryption (FHE) [18], and this advantage becomes more significant as the feature vector dimension increases.

2. We present the secure outsourced two-party inference framework built upon Vector-HSS, enabling a model owner and a user to jointly infer while preserving the privacy of both the model weights and the input. Our design leverages Vector-HSS's native support for high-dimensional inner products. We implement the full inference pipeline and demonstrate its feasibility: sub-second latency for 128×128 weight matrices and second-level performance at 512×512 scales, confirming Vector-HSS as a practical foundation for real-world privacy-preserving inference.

2 Technical Overview

In this section, we present a high-level overview of our contributions.

Prior HSS construction. Existing HSS schemes typically adopt Restricted Multiplication Straight-line (RMS) programs as their underlying computational model for evaluating branching programs. Unlike general arithmetic circuits, the RMS model enforces a strict constraint on multiplication: while it permits multiplying an input value by a computed intermediate value (also called memory values), it strictly prohibits multiplying two memory values directly. Generally, the RMS model consists of four fundamental operations:

- `ConvertInput`: Convert input values into memory values.
- `Add`: Sum two memory values or input values.
- `Mul`: Multiply an input value by a memory value.
- `Output`: Extract the final result.

Among these operations, `ConvertInput` is essential for enabling multiplications, as any input value intended to serve as a multiplier in subsequent computations must first be converted to a memory value. However, high-dimensional vector secure computation typically involves a large number of multiplications over vector elements, requiring most inputs to undergo `ConvertInput`. Consequently, frequent invocation of `ConvertInput` incurs substantial computational costs, significantly degrading execution efficiency.

By studying the existing HSS scheme [38], we find that the primary objective of `ConvertInput` is to generate two essential shares: $\langle x\rangle_\sigma$ and $\langle xv\rangle_\sigma$, which are required for secure multiplication. Here, x is a small integer over $\mathbb{Z}_N$ to prevent overflow errors, v is the system key sampled from $\mathbb{Z}^*_{N^2}$, and σ is in $\{0,1\}$.

The share $\langle x\rangle_\sigma$ can be computed by the server from the input value $I_x = (ct_0, ct_1) = \left((1+N)^x g^{uv},\ g^{-u}\right)$ and the memory value $M_{1,\sigma} = (\langle 1\rangle_\sigma, \langle v\rangle_\sigma)$ (a share of 1) using $\mathbf{DDLog}\left(ct_0^{\langle 1\rangle_\sigma} \cdot ct_1^{\langle v\rangle_\sigma}\right)$. Intuitively, computing $\langle xv\rangle_\sigma$ could follow a similar logic using an input value $I'_x = (ct'_0, ct'_1) = ((1+N)^x g^{u'}, g^{-u'v})$. Here, both u and u' are random numbers chosen from $\mathbb{Z}^*_N$. However, a fundamental obstacle arises from the modular nature of **DDLog**: its output is strictly constrained modulo N, while the system key v is sampled from $\mathbb{Z}^*_{N^2}$. As a result, the product xv exceeds N with overwhelming probability, causing modular overflow and leading to incorrect results if the multiplication is performed directly. To avoid overflow and ensure correctness, Orlandi et al. [38] adopt a key decomposition strategy:

1. Decompose the key v into ℓ small-base digits $(v^{(0)}, \ldots, v^{(\ell-1)})$.
2. Construct the input values
$$\left(g^{-uv^{(0)}}, \ldots, g^{-uv^{(\ell-1)}}, (1+N)^x g^u\right).$$
3. Apply the **DDLog** operation to each component and recover the final $\langle xv\rangle_\sigma$.

This complex decomposition needs multiple modular exponentiations and **DDLog** operations. In high-dimensional vector settings, this overhead accumulates rapidly, creating significant computational and communication burdens.

Our ConvertInput-Free Vector-HSS. Motivated by HSS schemes for RMS programs [38], we focus on efficient high-dimensional vector computation rather than general branching programs. A key limitation of existing approaches is their reliance on the costly ConvertInput operation. To overcome this, we propose ConvertInput-free Vector-HSS, a specialized HSS scheme that enables efficient linear vector computation without invoking ConvertInput.

In our framework, vector computations no longer require the server to invoke ConvertInput to generate vector shares $\langle \mathbf{x}\rangle_\sigma$ and key-bound shares $\langle \mathbf{x}[i]v\rangle_\sigma$ (for $i \in [1,d]$), as in traditional HSS schemes. Instead, we eliminate ConvertInput by adopting a restructured computation model, in which these shares are derived naturally from the client-side computation. Specifically, for a high-dimensional input vector $\mathbf{x}$, the share generation process remains straightforward. The client can simply split $\mathbf{x}$ into $\langle \mathbf{x}\rangle_\sigma$ ($\sigma \in \{0,1\}$) using subtractive secret sharing and securely distribute the shares to the corresponding servers. However, a challenge arises when attempting to generate shares of the product $\langle \mathbf{x}v\rangle_\sigma$, which binds the input with the system key v, since v is unknown on the client side.

A natural approach is to utilize the system public key $pk = g^v$ and construct

$$pk^{\mathbf{x}} = \left((g^v)^{\mathbf{x}[1]}, (g^v)^{\mathbf{x}[2]}, \ldots\right),$$

thereby achieving the binding between $\mathbf{x}$ and v. Unfortunately, this method is insecure. Because each element $\mathbf{x}[i]$ of $\mathbf{x}$ is typically a small integer, an adversary could recover $\mathbf{x}[i]$ via brute-force enumeration against $pk^{\mathbf{x}[i]}$.

To ensure security, we extend the system key from a scalar v to a key vector $\mathbf{v}$ and bind $\mathbf{x}$ with $\mathbf{v}$ through an exponent inner product $g^{\mathbf{x}\cdot\mathbf{v}}$ (The formal analysis in Theorem 4 proves that this construction achieves provable security). Then, we replace the memory value $M_\sigma = (\langle x\rangle_\sigma, \langle xv\rangle_\sigma)$ generated by `ConvertInput` in traditional HSS with $\mathbf{A}_\sigma = (\langle \mathbf{x}\rangle_\sigma, g^{\mathbf{x}\cdot\mathbf{v}})$ generated by the client.

Building on this new binding mechanism, we further revise the design of the evaluation key. In conventional schemes [1,30,38], each server S_σ ($\sigma \in \{0,1\}$) holds an evaluation key $\langle ek\rangle_\sigma$ represented by the share $\langle v\rangle_\sigma$ of the system key v. In contrast, in our construction, the evaluation key is replaced by the share $\langle u\rangle_\sigma$ of a random value u generated by the client. This modification aligns naturally with the new computation structure and leads to a simpler and more efficient server-side evaluation.

Based on the above design, we significantly reduce the computational cost of secure inner product computation. Previous

work computes

$$\mathbf{DDLog}(ct_0^{\langle x\rangle_\sigma} \cdot ct_1^{\langle xv\rangle_\sigma})$$

for scalar multiplication. When extended to a d-dimensional vector, the computation becomes

$$\mathbf{DDLog}(\prod_{i=1}^{d} \mathbf{ct}_0[i]^{\langle \mathbf{x}[i]\rangle_\sigma} \cdot \mathbf{ct}_1[i]^{\langle \mathbf{x}[i]v\rangle_\sigma}),$$

requiring $2d$ exponentiations. Our Vector-HSS reduces this to $d+1$ by using

$$\mathbf{DDLog}(\mathbf{A}_{\sigma,1}^{\langle u\rangle_\sigma} \cdot \prod_{i=1}^{d} \mathbf{ct}[i]^{\langle \mathbf{x}[i]\rangle_\sigma}),$$

nearly halving computational overhead, where $\mathbf{A}_{\sigma,1} = g^{\mathbf{x}\cdot\mathbf{v}}$ and $\mathbf{ct}[i] = (1+N)^{\mathbf{x}[i]} g^{u\mathbf{v}[i]}$.

Batch Operation. Although our Vector-HSS reduces the overhead of a single inner product computation, high computational cost still arises when inner product computations must be computed frequently and independently. Consider typical privacy-preserving image retrieval scenarios: the feature vector of a query image must be evaluated against numerous encrypted image vectors stored in the database. In this case, $d+1$ exponentiations must be performed on each query-database vector pair, resulting in significant computational cost for high-dimensional vectors.

To address this limitation, we incorporate CRT to design a batch processing mechanism, which packs multiple inner product computations and evaluates them within a single computation. Specifically, this mechanism allows the inner product computation between a fixed vector $\mathbf{x}$ and multiple vectors to be packaged into a single protocol execution. This batching method reduces a large number of repetitive exponential operations, thereby effectively improving overall computational efficiency.

From Primitive to Application. In Section 5, we design four generic vector secure computing modules based on the Vector-HSS, including secure Euclidean/cosine distance computation, secure comparison, and secure matrix-vector multiplication. Using these modules, we demonstrate two practical application scenarios of Vector-HSS in Section 6.

1. **Privacy-preserving Image Retrieval Based on Vector-HSS.** Compared with state-of-the-art approaches, our retrieval scheme achieves higher efficiency while maintaining strong security guarantees (for more details, see Table 3 in Section 7.2):

   - **vs. Secure KNN:** While Secure KNN [35, 41, 42] offers high efficiency, it is vulnerable to Ciphertext-Only Attacks (COA) [36]. Our scheme provides provable semantic security.
   - **vs. Paillier/MPC:** Schemes based on Paillier [45] and MPC [32] offer strong security but suffer from significant performance degradation with retrieval times typically on the hundreds of seconds. In Contrast, our scheme reduces retrieval time to hundreds of milliseconds while maintaining security, making it more suitable for large-scale retrieval.
   - **vs. FHE:** Retrieval time in our scheme achieves sub-second performance, significantly outperforming the second-level retrieval efficiency of the CKKS-based scheme [18]. Moreover, unlike FHE-based schemes that require the data owner to be online or share its secret key, our scheme does not require either, thus enabling more flexible deployment.

2. **Secure Outsourced Two-party Inference.** Built upon our matrix-vector multiplication protocol, our inference scheme allows a model owner holding $\mathbf{W}$ and a user holding $\mathbf{x}$ to compute $\mathbf{Wx}$ in outsourced settings, while protecting both the model and input vector. Experimental results confirm its practical efficiency, achieving sub-second inference for 128×128 dimensions matrix and maintaining second-level performance at 512×512 scales.

3 Preliminaries

3.1 Notation

Throughout this work, κ is a security parameter, and the notation $x \leftarrow_R \mathbb{Z}$ signifies that x is sampled uniformly at random from the group $\mathbb{Z}$.

In application scenarios, $d \in \mathbb{Z}$ represents vector dimension, and $l \in \mathbb{Z}$ is the amount of data that can be processed in a single batch. We use $\mathcal{N}$ to denote the total number of images on dataset. For the encryption part, pk denotes the system public key. We use ask to denote auxiliary information, and $\langle x\rangle$ denotes the secret sharing of x. Additionally, bold letters (such as $\mathbf{x}$) denote vectors. We use $\mathbf{x}_i$ to denote the i-th vector in a set, and $\mathbf{x}_i[j]$ to denote the j-th element of $\mathbf{x}_i$. We use $[\![\cdot]\!]$ to denote a Paillier [39] ciphertext. In the k-means tree structure, $\mathcal{T}_t$ denotes the t-th level of the tree, and $\mathcal{T}_t[j]$ denotes the j-th element in the t-th level.

3.2 Decisional Composite Residuosity Assumption

The Decisional Composite Residuosity (DCR) assumption concerns the computational hardness of distinguishing N-th residues modulo N^2 from uniformly random elements in $\mathbb{Z}_{N^2}^*$. It has been widely adopted in many foundational cryptographic structures. The DCR assumption is presented as follows:

Definition 1 (DCR Assumption)**.** *Given $N = pq$, where p and q are primes. For any PPT distinguisher $\mathcal{D}$, there exists a negligible function* $\mathrm{negl}(\cdot)$ *such that*

$$\left|\Pr[\mathcal{D}(N, z = z_0^N \bmod N^2) = 1] - \Pr[\mathcal{D}(N, u) = 1]\right| \leq \mathrm{negl}(\kappa),$$

where $z_0 \xleftarrow{R} \mathbb{Z}_N^*$, and $u \xleftarrow{R} \mathbb{Z}_{N^2}^*$. In other words, it is hard in polynomial time to distinguish an N-th residue modulo N^2 and a uniformly random element in $\mathbb{Z}_{N^2}^*$ [44].

3.3 Distributed Discrete Log

Orlandi et al. [38] introduced DDLog, a deterministic share conversion operation that transforms multiplicative (division) shares into additive (subtraction) shares in the Paillier group. Concretely, suppose two parties hold group elements $g_0, g_1 \in \mathbb{Z}_{N^2}^*$ such that $\frac{g_1}{g_0} = (1+N)^x \bmod N^2$. Each party locally applies the DDLog procedure to its share g_σ ($\sigma \in 0,1$) to obtain $z_\sigma = \mathbf{DDLog}(g_\sigma)$, where $\mathbf{DDLog}(g)$ computes $h = g \bmod N$ and $h' = \lfloor g/N \rfloor$, then outputs $z = h' \cdot h^{-1} \bmod N$. The resulting values satisfy $z_1 - z_0 = x \bmod N$, thereby yielding additive shares of the exponent x and enabling its correct recovery with perfect correctness.

This conversion is perfectly correct for any $x \in \mathbb{Z}_N$.

3.4 Chinese Remainder Theorem

The principle of the Chinese Remainder Theorem (CRT) is described as follows:

Given a set of pairwise coprime positive integers $\{\psi_1, \psi_2, \ldots, \psi_l\}$, the system of congruences $m \equiv m_i \pmod{\psi_i}$ $(1 \leq i \leq l)$ has a unique solution modulo $\Psi = \prod_{i=1}^{l} \psi_i$, which can be expressed as: $m \equiv \left(\sum_{i=1}^{l} m_i \cdot \Phi_i\right) \bmod \Psi$, where $\Psi_i = \Psi/\psi_i$ and $\Phi_i = \Psi_i \cdot (\Psi_i^{-1} \bmod \psi_i)$.

In computer science, CRT enables efficient computation by decomposing large integer operations into multiple independent operations on residue vectors:

1. **Vector Addition:**

   $\mathbf{A} + \mathbf{B} = ((A_1 + B_1) \bmod \psi_1, \ldots, (A_l + B_l) \bmod \psi_l)$;

2. **Vector Multiplication:**

   $\mathbf{A} \times \mathbf{B} = ((A_1 \cdot B_1) \bmod \psi_1, \ldots, (A_l \cdot B_l) \bmod \psi_l)$.

This decomposition allows computations to be performed on a smaller modulus, thereby reducing computational complexity and improving efficiency.

4 Vector Homomorphic Secret Sharing

In this section, we begin by formalizing the definition of Vector-HSS, which combines the notion of HSS [38]. We then describe the concrete construction and finally provide a rigorous correctness and security analysis of Vector-HSS.

4.1 Definition

Definition 2 (Vector-HSS)**.** *Let $\mathcal{P}$ be a class of programs over a ring R, a 2-party public-key HSS scheme for $\mathcal{P}$ consists of four algorithms (`Gen`, `CGen1`, `CGen2`, `Eval`), described as follows:*

- $\texttt{Gen}(1^\kappa) \to pk$*: Taking the security parameter 1^κ as input, the algorithm outputs a set of public keys pk.*
- $\texttt{CGen1}(pk, \boldsymbol{x}) \to (\boldsymbol{A}_0, \boldsymbol{A}_1)$*: Given pk and a private vector $\boldsymbol{x} \in R^d$, the client runs the algorithm to output the auxiliary input values $(\boldsymbol{A}_0, \boldsymbol{A}_1)$.*
- $\texttt{CGen2}(pk, \boldsymbol{y}) \to ((\langle ek \rangle_0, \langle ek \rangle_1), \boldsymbol{I})$*: Given pk and a primary vector $\boldsymbol{y} \in R^d$, the client runs the algorithm to output a primary input value $\boldsymbol{I}$ and a pair of evaluation keys $(\langle ek \rangle_0, \langle ek \rangle_1)$ such that $ek = \langle ek \rangle_1 - \langle ek \rangle_0$.*
- $\texttt{Eval}(\sigma, ek_\sigma, \boldsymbol{A}_\sigma, \boldsymbol{I}, P) \to z_\sigma$*: Server S_σ runs the algorithm to perform the homomorphic evaluation, where $\sigma \in \{0,1\}$. It accepts an auxiliary input value $\boldsymbol{A}_\sigma$, an evaluation key ek_σ, a primary input value $\boldsymbol{I}$, and a target program $P \in \mathcal{P}$. It outputs a share z_σ. The final result can be recovered from these shares.*

This Vector-HSS scheme must satisfy: ***Correctness:*** *For any security parameter $\kappa \in \mathbb{N}$, program $P \in \mathcal{P}$, inputs $\boldsymbol{x}$ and $\boldsymbol{y}$, there exists a negligible function* $\mathrm{negl}(\kappa)$ *satisfying*

$$\Pr\left[\begin{array}{c|c} z_0 + z_1 \neq P(\boldsymbol{x}, \boldsymbol{y}) & \begin{array}{c} pk \leftarrow \mathrm{Gen}(1^\kappa) \\ (\boldsymbol{A}_0, \boldsymbol{A}_1) \leftarrow \mathrm{CGen1}(pk, \boldsymbol{x}) \\ ((\langle ek \rangle_0, \langle ek \rangle_1), \boldsymbol{I}) \leftarrow \mathrm{CGen2}(pk, \boldsymbol{y}) \\ z_\sigma = \mathrm{Eval}(\sigma, \langle ek \rangle_\sigma, \boldsymbol{A}_\sigma, \boldsymbol{I}, P) \end{array} \end{array}\right] \leq \mathrm{negl}(\kappa),$$

where $\sigma \in \{0,1\}$.

Security (via Simulation): *Let $\mathcal{A}_{S_\sigma}$ be a PPT adversary (i.e., semi-honest server S_σ), and $\mathcal{S}_{S_\sigma}$ be its simulator, where $\sigma \in \{0,1\}$. The Vector-HSS protocol Π is secure if for all $\mathbf{A}_\sigma$, ciphertext sequences $\mathbf{I}$, and result shares z_σ, the view REAL of $\mathcal{A}_{S_\sigma}$ in the real world and the view SIM of $\mathcal{S}_{S_\sigma}$ in the ideal world are computationally indistinguishable:*

$$REAL^{\Pi}_{\mathcal{A}_{S_\sigma}}(\mathbf{A}_\sigma, \mathbf{I}, z_\sigma) \approx_c SIM_{\mathcal{S}_{S_\sigma}}(\mathbf{A}_\sigma, \mathbf{I}, z_\sigma),$$

where $\approx_c$ denotes computational indistinguishability.

4.2 Construction of Vector-HSS

Unlike the scalar-based setting considered in [38], our scheme takes vectors as input. More importantly, our construction eliminates the `ConvertInput` operation by the design in Section 2. Specifically, for a vector $\mathbf{x}$:

- The primary input value is

$$\mathbf{I}_\mathbf{x} = \left\{(g^{v_i})^{ek_\mathbf{x}} \cdot (1+N)^{\mathbf{x}[i]}\right\}_{i=1}^{d}.$$

- The auxiliary input value for party σ is $\mathbf{A}_{\mathbf{x},\sigma} = (\langle \mathbf{x} \rangle_\sigma, ask_\mathbf{x})$, consisting of subtractive shares $\langle \mathbf{x} \rangle_\sigma$ and auxiliary information $ask_\mathbf{x} = g^{\sum_{i=1}^{d} v_i x_i}$.

Our Vector-HSS scheme for high-dimensional vectors is described in Fig. 1 and its key evaluation operations include vector addition (`Add`), vector inner product (`IP`), and batched vector inner product (`BIP`). Due to the inherent properties of HSS, vector addition can be regarded as a direct generalization of scalar addition on d-dimensional vectors. Therefore, additional correctness or security proofs are unnecessary. Here, we primarily present the correctness and security proofs for the inner product computation in Section 4.3.

Note that in Fig. 1, parameters $\{\psi_j\}_{j=1}^{l}$ and $\{\phi_j\}_{j=1}^{l}$ of CRT are a set of l small primes satisfying the following conditions:

$$\begin{cases} \mathbf{x}[i] \le m & (\forall 1 \le i \le d) \\ \mathbf{y}[i] \le m & (\forall 1 \le i \le d) \\ 2d \cdot m^2 \le \psi_j & (\forall 1 \le j \le l) \\ 2d \cdot m \cdot (\phi_1 \cdot m + \phi_2 \cdot m + \cdots + \phi_l \cdot m) \le N \end{cases} \tag{1}$$

where m is a positive integer denoting the upper bound on the plaintext values and $|m|$ denotes the bit length of m.

Discussion: The Vector-HSS scheme in this paper is based on the semi-honest model, i.e., it assumes that both the servers and the client will honestly follow the protocol. Moreover, the scheme has the potential to be extended to the malicious model. A natural approach for such an extension is to introduce zero-knowledge proofs, but even with the highly efficient known techniques [33], the cost of generating a single proof would exceed that of the entire work. Therefore, how to design an extension scheme suitable for the malicious model without sacrificing efficiency will be a key focus of our future work.

4.3 Proof of Correctness and Security

The proofs of Theorem 1 - 4 are provided in Appendix A due to space constraints.

Theorem 1. *The IP operation and its batched extension BIP in Fig. 1 correctly compute the inner product shares.*

Theorem 2. *Let $N = pq$ for distinct primes p, q. If*

$$d > \frac{(r+1)\max\{\log p, \log q\} + \omega(\log r)}{|m|}, \tag{2}$$

and C is uniform in $\{-2^{|m|-1}, \ldots, 2^{|m|-1}-1\}^{d \times c}$, then for uniform matrices $E \in \mathbb{Z}_N^{r \times d}$, $B \in \mathbb{Z}_N^{r \times c}$ and any vector $w \in \mathbb{Z}_q^d$, the joint distribution $(E, EC, C^\mathsf{T} w)$ is statistically indistinguishable from $(E, B, C^\mathsf{T} w)$.

Theorem 3. *Let T_1 and T_2 be positive integers, and let $T = T_1 + T_2$. Let u be an integer uniformly random in the interval $[0, T-1]$. Let P be the distribution of $(u \bmod T_1)$, and $U_{\mathbb{Z}_{T_1}}$ be the uniform distribution over $\mathbb{Z}_{T_1}$. Then the statistical distance between them satisfies:*

$$\mathbf{SD}(P, U_{\mathbb{Z}_{T_1}}) \le \frac{T_2}{T_1},$$

where $\mathbf{SD}(X, Y)$ denotes the statistical distance between distributions X and Y. In particular, when T_1 is sufficiently large and $T_2 = o(T_1)$, this distance is negligible.

Theorem 4. *Consider any PPT adversary $\mathcal{A}_{S_\sigma}$ and its simulator $\mathcal{S}_{S_\sigma}$ in the security experiment defined in 2, where $\sigma \in \{0,1\}$. The inner product operation in protocol Π securely computes the inner product functionality if for all auxiliary input value $\mathbf{A}_{\mathbf{x},\sigma}$, primary input value $\mathbf{I}_\mathbf{y}$ ($\mathbf{x}[i], \mathbf{y}[i] \in [-2^{|m|-1}, 2^{|m|-1}-1]$) and result share z_σ, the view REAL of $\mathcal{A}_{S_\sigma}$ in the real world and the view SIM of $\mathcal{S}_{S_\sigma}$ in the ideal world are computationally indistinguishable:*

$$REAL^{\Pi}_{\mathcal{A}_{S_\sigma}}(\mathbf{A}_{\mathbf{x},\sigma}, \mathbf{I}_\mathbf{y}, z_\sigma) \approx_c SIM_{\mathcal{S}_{S_\sigma}}(\mathbf{A}_{\mathbf{x},\sigma}, \mathbf{I}_\mathbf{y}, z_\sigma),$$

where $\approx_c$ denotes computational indistinguishability.

5 Generic Vector Secure Computing Modules

Inner product is a core operation in high-dimensional vector computation. In principle, it directly supports Euclidean/cosine distance computation and matrix-vector multiplication. However, in practical applications, these computations are often intermediate results for subsequent comparisons, rankings, or inferences, rather than final outputs. Motivated by this, we design a set of distance-related secure computation protocols, which can be used together with our secure comparison protocol for distance-based secure comparison scenarios (such as PIR, biometric authentication, and recommender systems). Additionally, we construct a secure matrix-vector multiplication protocol to support linear-algebra-based privacy-preserving tasks. Due to the limitation of space, the correctness and security proofs of our protocols are provided in Appendix B and C, respectively.

5.1 Secure Distance-related Protocol

In many distance-based comparison computation tasks involving high-dimensional vectors, a common pattern is to compute the distances between a given vector and several different vectors.

Euclidean Distance. The square form of the Euclidean distance can be expressed as

$$edist_i = \|\mathbf{x}_i - \mathbf{y}\|^2 = \langle \mathbf{x}_i, \mathbf{x}_i \rangle - 2\langle \mathbf{x}_i, \mathbf{y} \rangle + \langle \mathbf{y}, \mathbf{y} \rangle. \tag{3}$$

In fact, we could compute $edist_i$ directly by computing three terms on the right in Eq. 3. However, observation reveals that for any i, Eq. 3 contains a common term $\langle \mathbf{y}, \mathbf{y} \rangle$, so it can be

$\texttt{Gen}(1^\kappa)$:

1. Generate two safe prime numbers $p = 2p' + 1$ and $q = 2q' + 1$ where p' and q' are sufficiently large primes satisfying $p', q' > 2^{\text{poly}(\kappa)}$ (with poly denoting a polynomial), and then compute $N = pq$.
2. Sample an element $g' \leftarrow_R \mathbb{Z}^*_{N^2}$ and an integer vector $\mathbf{v} = (v_1, \ldots, v_d)$, where $v_i \leftarrow_R \{1, \ldots, \lfloor N/4 \rfloor\}$. These values are then used to compute: $g = g'^{2N} \bmod N^2$, $\{h_i = g^{v_i} \bmod N^2\}_{i\in[1,d]}$.
3. Output $pk = (g, \{h_i\}_{i\in[1,d]}, N)$.

$\texttt{CGen1}(pk, \mathbf{x})$:

1. Split each element $\mathbf{x}[i]$ of $\mathbf{x}$ into two random shares $\langle \mathbf{x}[i]\rangle_0, \langle \mathbf{x}[i]\rangle_1 \leftarrow_R [0, 2^{|\mathbf{x}[i]|+1+\kappa}]$, such that $\langle \mathbf{x}[i]\rangle_0 - \langle \mathbf{x}[i]\rangle_1 = \mathbf{x}[i]$, where $|\mathbf{x}[i]|$ denotes the bit length of $\mathbf{x}[i]$. Then compute $ask_\mathbf{x} = \prod_{i=1}^d h_i^{\mathbf{x}[i]} \bmod N^2$.
2. Output $\mathbf{A}_{\mathbf{x},\sigma} = (\langle \mathbf{x}\rangle_\sigma, ask_\mathbf{x})$ to the corresponding server, where $\langle \mathbf{x}\rangle_\sigma$ is transmitted securely.

$\texttt{CGen2}(pk, \mathbf{y})$:

1. Select a random number $ek_\mathbf{y}$, compute $\mathbf{I}_\mathbf{y} = \left\{h_i^{ek_\mathbf{y}} \cdot (1+N)^{\mathbf{y}[i]}\right\}_{i=1}^d$ and split $ek_\mathbf{y}$ into $(\langle ek_\mathbf{y}\rangle_0, \langle ek_\mathbf{y}\rangle_1)$ with $ek_\mathbf{y} = \langle ek_\mathbf{y}\rangle_1 - \langle ek_\mathbf{y}\rangle_0$.
2. Output $(\langle ek_\mathbf{y}\rangle_\sigma, \mathbf{I}_\mathbf{y})$ to the corresponding server, where $\langle ek_\mathbf{y}\rangle_\sigma$ is transmitted securely.

$\texttt{Add}_\mathbf{A}(\sigma, \mathbf{A}_{\mathbf{x},\sigma}, \mathbf{A}_{\mathbf{y},\sigma})$:

1. S_σ holds two auxiliary input values $\mathbf{A}_{\mathbf{x},\sigma} = (\langle \mathbf{x}\rangle_\sigma, ask_\mathbf{x})$ and $\mathbf{A}_{\mathbf{y},\sigma} = (\langle \mathbf{y}\rangle_\sigma, ask_\mathbf{y})$.
2. Output $\mathbf{A}_{\mathbf{z},\sigma} = (\langle \mathbf{z}\rangle_\sigma, ask_\mathbf{z}) = (\langle \mathbf{x}\rangle_\sigma + \langle \mathbf{y}\rangle_\sigma, ask_\mathbf{x} \cdot ask_\mathbf{y})$.

$\texttt{Add}_\mathbf{I}(\langle ek_\mathbf{x}\rangle_\sigma, \langle ek_\mathbf{y}\rangle_\sigma, \mathbf{I}_\mathbf{x}, \mathbf{I}_\mathbf{y})$:

1. Perform vector addition by computing the primary input value product $\mathbf{I}_\mathbf{z} = \mathbf{I}_\mathbf{x} \cdot \mathbf{I}_\mathbf{y}$.
2. The output includes the updated evaluation key $\langle ek_\mathbf{z}\rangle_\sigma$. If the vectors are from different clients, $\langle ek_\mathbf{z}\rangle_\sigma = \langle ek_\mathbf{x}\rangle_\sigma + \langle ek_\mathbf{y}\rangle_\sigma$; otherwise, if from the same client, $\langle ek_\mathbf{z}\rangle_\sigma = \langle ek_\mathbf{x}\rangle_\sigma$ (no update required). The final output is $(\langle ek_\mathbf{z}\rangle_\sigma, \mathbf{I}_\mathbf{z})$.

- Let $\mathcal{B}_S^{(\sigma)} = (\mathbf{I}_\mathbf{x}, \mathbf{A}_{\mathbf{y},\sigma}, \langle ek_\mathbf{x}\rangle_\sigma)$ and $\mathcal{B}_M^{(\sigma)} = (\mathbf{I}_\mathbf{x}, \{\mathbf{A}_{j,\mathbf{y},\sigma}\}_{j=1}^l, \langle ek_\mathbf{x}\rangle_\sigma, \{\psi_j\}_{j=1}^l, \{\phi_j\}_{j=1}^l)$ denote the single-vector and multi-vector batch tuples for S_σ, respectively, where l is the size of a batch. In these tuples, for all $1 \le j \le l$, we define $\Psi_j = \Psi/\psi_j$ and $\phi_j = \Psi_j \cdot (\Psi_j^{-1} \bmod \psi_j)$, such that ψ_j and ϕ_j satisfy the condition in Eq. 1.

$\texttt{IP}(\mathcal{B}_S^{(\sigma)})$:

1. Compute the homomorphic inner product parameters: $(H_\sigma \leftarrow \prod_{i=1}^d \mathbf{I}_\mathbf{x}[i]^{\langle \mathbf{y}[i]\rangle_\sigma} \bmod N^2, D_\sigma \leftarrow ask_\mathbf{y}^{\langle ek_\mathbf{x}\rangle_\sigma} \bmod N^2)$.
2. Perform distributed discrete log: $z_\sigma \leftarrow \mathbf{DDLog}(D_\sigma \cdot H_\sigma)$. If $z_\sigma > N/2$, then S_σ computes $z_\sigma = z_\sigma - N$.
3. Output z_σ.

$\texttt{BIP}(\mathcal{B}_M^{(\sigma)})$:

1. Perform batch operations: $(\langle \mathbf{Y}\rangle_\sigma \leftarrow \sum_{j=1}^l \phi_j \cdot \langle \mathbf{y}[j]\rangle_\sigma, ASK \leftarrow \prod_{j=1}^l ask_{j,\mathbf{y}}^{\phi_j} \bmod N^2)$.
2. Compute the homomorphic inner product parameters: $(H'_\sigma \leftarrow \prod_{i=1}^d \mathbf{I}_\mathbf{x}[i]^{\langle \mathbf{Y}[i]\rangle_\sigma}, D'_\sigma \leftarrow ASK^{\langle ek_\mathbf{x}\rangle_\sigma} \bmod N^2)$.
3. Perform distributed discrete log: $z'_\sigma \leftarrow \mathbf{DDLog}(D'_\sigma \cdot H'_\sigma)$. The list Z_σ is generated by reducing z'_σ modulo each ψ_j and centering the result into $[-\psi_j/2, \psi_j/2]$.
4. Output Z_σ.

Figure 1: Construction of Vector-HSS

eliminated during distance comparison. That is, the comparison can be completed simply by calculating $\langle \mathbf{x}_i, \mathbf{x}_i \rangle - 2\langle \mathbf{x}_i, \mathbf{y} \rangle$, which can be efficiently computed through inner product operations. Based on this observation, we propose a secure Euclidean distance-related sharing protocol based on Vector-HSS, as shown in Protocol 1.

Notably, red parts in this protocol represent an additional operation specific to the privacy-preserving image retrieval scenario (see Fig. 2 in Section 6.1). Specifically, $\mathbf{x}_i$ in Fig. 2 represents the sum of all vectors within a cluster, so the sum needs to be normalized according to the cluster size $|\mathcal{E}_i|$ before distance calculation. Removing these parts does not affect the correctness of protocol 1. The normalization achieved using Eq. (4):

$$\langle \text{avgyx}_i \rangle_\sigma = \langle \langle \mathbf{x}_i \rangle_\sigma, \mathbf{y} \rangle \cdot \mathbf{Appr}\left(1/|\mathcal{E}_i|\right) \tag{4}$$

where $\mathbf{Appr}(\cdot)$ denotes a fixed-point approximation mapping. It maps floating-point numbers to the integer ring $\mathbb{Z}_{deg}$ via scaling and rounding according to $\mathbf{Appr}(m) = \lfloor m \cdot deg \rceil$, where deg is the precision parameter and $\lfloor \cdot \rceil$ is the nearest-integer rounding operation.

Protocol 1 Secure Euclidean Distance-related Sharing Protocol (SEDS) Base on Vector-HSS

Input: A batched data tuple $\mathcal{B}_M^{(\sigma)}$, cluster sizes $\{|\mathcal{E}_i|\}_{i=1}^l$, and a set of subtractive shares of the sum of squared vectors $\{\langle avgx_i^2 \rangle_\sigma\}_{i=1}^l$ where $\sigma \in \{0,1\}$

Output: Shares of the variant Euclidean distance for each cluster $\langle \mathsf{dist} \rangle_\sigma$

Server S_σ computation:
$\{\langle \mathbf{x}_i, \mathbf{y} \rangle_\sigma\}_{i=1}^l \leftarrow \mathtt{BIP}(\mathcal{B}_M^{(\sigma)})$
for $i = 1$ to l **do**
  $inv_i \leftarrow \mathbf{Appr}\left(\frac{1}{|\mathcal{E}_i|}\right)$
  $\langle avgyx_i \rangle_\sigma \leftarrow \langle \mathbf{x}_i, \mathbf{y} \rangle_\sigma \times inv_i \times deg$
  $\langle dist_i \rangle_\sigma \leftarrow \langle avgx_i^2 \rangle_\sigma - 2 \cdot \langle avgyx_i \rangle_\sigma$
  Append $\langle dist_i \rangle_\sigma$ to the list $\langle \mathsf{dist} \rangle_\sigma$
end for
return $\langle \mathsf{dist} \rangle_\sigma$

Cosine Distance. The cosine distance can be expressed as:

$$cdist_i = 1 - \frac{\langle \mathbf{x}_i, \mathbf{y} \rangle}{\|\mathbf{x}_i\| \|\mathbf{y}\|}. \tag{5}$$

Similar to the Euclidean distance, the constant term 1 can be eliminated during distance comparison. For the latter term, its computation can essentially be reduced to directly calculating the inner product of the two already normalized vectors. Based on this observation, we propose a secure variant cosine distance-related sharing protocol based on Vector-HSS, as shown in Protocol 2.

It is worth noting that the red parts in this protocol also represent an additional operation specific to the retrieval algorithm in Fig. 2. In this algorithm, the query vector $\mathbf{y}$ can be normalized by the query user, while the squared 2-norm of the cluster center vectors $\mathbf{x}_i$ is not directly accessible to the server. Therefore, servers needs to pre-compute $\|\mathbf{x}_i\|^2$ to normalize $\mathbf{x}_i$.

Protocol 2 Secure Cosine Distance-related Sharing Protocol (SCDS) Base on Vector-HSS

Input: Given $\{\|\mathbf{x}_i\|^2\}_{i=1}^l$ and a batched data tuple $\mathcal{B}_M^{(\sigma)}$ where $\sigma \in \{0,1\}$

Output: Shares of the variant Cosine distance for each cluster $\langle \mathsf{dist} \rangle_\sigma$

Server S_σ computation:
$\{\langle \mathbf{x}_i, \mathbf{y} \rangle_\sigma\}_{i=1}^l \leftarrow \mathtt{BIP}(\mathcal{B}_M^{(\sigma)})$
for $i = 1$ to l **do**
  $cos_i = \langle \mathbf{x}_i \cdot \mathbf{y} \rangle_\sigma \times \mathbf{Appr}\left(\sqrt{\frac{1}{\|\mathbf{x}_i\|^2}}\right)$
  $\langle dist_i \rangle_\sigma \leftarrow -cos_i$
  Append $\langle dist_i \rangle_\sigma$ to the list $\langle \mathsf{dist} \rangle_\sigma$
end for
return $\langle \mathsf{dist} \rangle_\sigma$

5.2 Secure Comparison Protocol

Inspired by [47], we design a general secure comparison protocol in a two-party HSS setting. We use this protocol in the retrieval application shown in Fig. 2 to evaluate which cluster center $\mathbf{x}_i$ is closer to the query vector $\mathbf{y}$. In practice, this protocol can be generalized to secure comparison in any two-party HSS setting.

In this protocol, S_0 initiates the comparison by sending Paillier-encrypted values blinded by a random bit π. Then, S_1 processes these with its shares and random masks. Finally, S_0 decrypts the response to obtain bit μ, where $\pi \oplus \mu$ reveals the closer cluster center. Details are provided in Protocol 3.

It is worth mentioning that, based on this comparison protocol, we can design a Secure Minimum (SM) protocol using the binary search method from [46]. Since this idea is straightforward, we omit the detailed algorithm here; however, it will later be utilized in the image retrieval application shown in Fig. 2.

5.3 Matrix-Vector Multiplication Protocol

By partitioning the matrix into blocks based on row vectors and combining batch inner product operations in Vector-HSS, we design a matrix-vector multiplication module. In this protocol, each server iteratively processes the share of the matrix in batches. For the i-th batch, the server constructs $\mathbf{A}_{i,\sigma}$ and $\mathcal{B}_M^{(\sigma)}$ to compute the share $\langle \mathbf{z}_i \rangle_\sigma$ of a sub-vector. The shares of all sub-vectors from the server are concatenated to form the share $\langle \mathbf{Z} \rangle_\sigma$ of the final result vector. The detailed protocol is

presented in Protocol 4. Furthermore, this design can be naturally extended to support secure matrix-matrix multiplication.

Protocol 3 Secure Comparison Protocol (SC)

Input: Server S_0 holds subtractive shares $\langle dist_1\rangle_0$ and $\langle dist_2\rangle_0$; Server S_1 holds subtractive shares $\langle dist_1\rangle_1$ and $\langle dist_2\rangle_1$; Parameters: κ (e.g., $\kappa = 128$), N_P (Paillier modulus).

Output: Comparison result indicates whether $dist_1 \leq dist_2$ or not.

Server S_0:
Randomly select $\pi \in \{0,1\}$
if $\pi = 0$ **then**
  Compute $[\![\xi]\!] = [\![\langle dist_1\rangle_0 - \langle dist_2\rangle_0 + 1]\!]$ and $[\![-1]\!]$
  Send $([\![\xi]\!], [\![-1]\!])$ to S_1
else
  Compute $[\![\xi]\!] = [\![\langle dist_2\rangle_0 - \langle dist_1\rangle_0]\!]$ and $[\![1]\!]$
  Send $([\![\xi]\!], [\![1]\!])$ to S_1
end if
Server S_1:
Randomly select $r_1 \in \{0,1\}^\kappa \setminus \{0\}$ and r_2 such that $r_2 \leq \frac{N_P}{2}$ and $r_1 + r_2 > \frac{N_P}{2}$
Receive $([\![\xi]\!], [\![\pm 1]\!])$ from S_0
Compute: $[\![\zeta]\!] = [\![\xi]\!]^{r_1} \cdot [\![\pm 1]\!]^{r_1 \cdot (\langle dist_1\rangle_1 - \langle dist_2\rangle_1)} \cdot [\![r_2]\!]$
Send $[\![\zeta]\!]$ to S_0
Server S_0:
Receive $[\![\zeta]\!]$ and decrypt it to obtain ζ
Determine μ where: $\mu = 0$, if $\zeta \in \mathbb{Z}_N^-$; $\mu = 1$, if $\zeta \in \mathbb{Z}_N^+$.
Calculate $\pi \oplus \mu$ to compare $dist_1$ and $dist_2$: $dist_1 < dist_2$, if $\pi \oplus \mu = 1$; $dist_1 \geq dist_2$, if $\pi \oplus \mu = 0$

6 Applications

6.1 Privacy-Preserving Image Retrieval

Private information retrieval (PIR) typically refers to retrieving data from an encrypted database without revealing the query content [21], while privacy-preserving image retrieval, as a specific case of PIR, is designed for querying encrypted images. Queries in image retrieval are typically performed using feature vectors. Therefore, our Vector-HSS is particularly suitable for this application scenario. Based on protocols designed in Section 5, we have developed a privacy-preserving multi-layer clustering image retrieval algorithm.

Fig. 2 illustrates the overall architecture of the proposed algorithm, which involves two non-colluding servers, $\mathbf{n}_i$ image owners (IOs), $\mathbf{n}_q$ query users (QUs), and a key generation center (KGC). For simplicity, we set $\mathbf{n}_q = 1$ in Fig. 2. The KGC generates the system public key pk during initialization. Each IO holds an image dataset $\mathcal{M}_u$ and feature vectors of each image. Firstly, the raw image data and feature vectors (initially floating-point values) are mapped to the integer ring $\mathbb{Z}_{deg}$ via

Protocol 4 Matrix Vector Multiplication Protocol (MVM) Based on Vector-HSS

Input: Share of $r \times c$ matrix $\langle \mathbf{W}\rangle_\sigma$, $\{ask\}_{i=1}^r$, vector ciphertext $\mathbf{I_x}$, an evaluation key $\langle ek\rangle_\sigma$, size of a batch l, parameters $(\{\psi_j\}_{j=1}^l, \{\phi_j\}_{j=1}^l)$

Output: Result vector share $\langle \mathbf{Z}\rangle_\sigma$

Server S_σ computation:
for $k = 1$ to $\lceil \frac{r}{l} \rceil$ **do**
  Bind the l row vectors selected from $\langle \mathbf{W}\rangle_\sigma$ with their corresponding ask to generate $\{\mathbf{A}_{i,\sigma}\}_{i=(k-1)l+1}^{kl}$
  $\mathcal{B}_M^{(\sigma)} = (\mathbf{I_x}, \{\mathbf{A}_{i,\sigma}\}_{i=(k-1)l+1}^{kl}, \langle ek_{\mathbf{x}}\rangle_\sigma, \{\psi_j\}_{j=1}^l, \{\phi_j\}_{j=1}^l)$
  $\langle \mathbf{z}_k\rangle_\sigma \leftarrow \mathtt{BIP}(\mathcal{B}_M^{(\sigma)})$
  Append $\langle \mathbf{z}_k\rangle_\sigma$ to the list $\langle \mathbf{Z}\rangle_\sigma$
end for
return $\langle \mathbf{Z}\rangle_\sigma$

a fixed-point mapping $\mathbf{Appr}(m) = \lfloor m \cdot deg \rceil$, where deg is the precision parameter. Next, IOs split their image matrices and feature vectors into shares, which are securely sent to the servers along with the evaluation key ($\langle ek\rangle_\sigma$). Meanwhile, ciphertexts ($\mathbf{I_f}$) of feature vectors and auxiliary information (ask) are also submitted to servers. After uploading, IOs can go offline. Subsequently, servers and query users execute a privacy-preserving multi-layer clustering image retrieval algorithm.

The first phase is implemented by servers independently to find the cluster center of every layer and construct a hierarchical cluster tree $\{\mathcal{T}_t\}$. The second phase runs between the servers and QU, which implements privacy-preserving image retrieval based on the hierarchical cluster tree. A QU generates a query vector to initiate a retrieval request. Depending on the precision requirements, we support two modes: fuzzy retrieval and exact retrieval.

6.2 Secure Outsourced Two-party Inference

Privacy-preserving machine learning (PPML) aims to ensure the data privacy of participants during model training and inference. Secure two-party inference, as PPML's core component, provides the foundation for achieving this goal. it enables two mutually distrustful parties, typically a cloud service provider holding a private model and a user holding private input data, to collaboratively complete inference tasks without revealing their private information [29, 31]. With the rise of cloud computing, more and more data holders intend to outsource complex inference tasks to the cloud, leveraging its powerful computing resources and reducing local computational burden. Since model inference typically relies on large-scale matrix-vector multiplication, our Vector-HSS is particularly well-suited for this scenario. Based on the matrix-vector protocol proposed in Section 5, we construct a secure

Input:

1. **System Parameters:** Security parameter κ, public key $pk = (g, \{h_j\}_{j=1}^{d}, N)$ generated by KGC, $\sigma \in \{0,1\}$.
2. **Server State:** For each image $\mathbf{img}_i$, server S_σ holds image matrix and corresponding feature vector shares $\{\langle \mathbf{img}_i \rangle_\sigma, \langle \mathbf{f}_i \rangle_\sigma\}$, encrypted feature vectors $\mathbf{I}_{\mathbf{f}_i}$, evaluation keys $\langle ek_i \rangle_\sigma$, and auxiliary information ask_i.
3. **Configuration:** Clustering threshold t, top-K parameter.

Phase I: Offline Secure Multi-Layer Clustering

1. **Initialization.** Server S_0 randomly selects l values as initial centers. After one iteration, it partitions the dataset indices DB into l clusters $\{\mathcal{E}_k\}_{k=1}^{l}$.
2. **Iterative Clustering.** Servers iterate until convergence:
   (a) *Secure Centroid Computation:* For each cluster $\mathcal{E}_k$, servers aggregate the shares $\{\langle \mathbf{f}_i \rangle_\sigma\}$, encrypted vectors $\{\mathbf{I}_{\mathbf{f}_i}\}$, evaluation keys $\{\langle ek_{\mathbf{f}_i} \rangle_\sigma\}$, and auxiliary keys $\{ask_{\mathbf{f}_i}\}$ to obtain the centroid representation $(\langle \mathbf{x}_k \rangle_\sigma, \mathbf{I}_{\mathbf{x}_k}, \langle ek_{\mathbf{x}_k} \rangle_\sigma, ask_{\mathbf{x}_k})$. Servers then invoke the inner product operation to compute the squared norm share $\langle \|\mathbf{x}_k\|^2 \rangle_\sigma$, and derive the average squared norm share $\langle avgx_k^2 \rangle_\sigma$.
   (b) *Secure Assignment Update:* For each feature vector $\mathbf{f}_i$, servers invoke the `SEDS` (or `SCDS`) protocol to compute distance shares to all cluster centroids. Servers then execute the `SM` protocol to determine the new cluster assignment without revealing actual distances.
   (c) *Convergence Check:* If the number of changed assignments falls below ε, the iteration terminates. Otherwise, servers update cluster assignments and continue.
3. **Recursive Partitioning.** After convergence, for each cluster $\mathcal{E}_k$:
   - If $|\mathcal{E}_k| > t$: servers extract the data belonging to cluster $\mathcal{E}_k$ from database DB to form the sub-dataset DB'_k and recursively apply the clustering algorithm to generate sub-clusters at hierarchy level $h+1$.
   - If $|\mathcal{E}_k| \leq t$: the cluster becomes a leaf node in the tree $\mathcal{T}_t[k] \leftarrow \mathcal{E}_k$.

Output: A hierarchical cluster tree $\{\mathcal{T}_t\}$ stored distributively on servers, where all leaf nodes satisfy size $\leq t$.

Phase II: Online Privacy-Preserving Retrieval

1. **Query Submission.** QU extracts and encrypts the query vector $\mathbf{y}$ to obtain $\mathbf{I}_\mathbf{y}$. It then generates evaluation key shares $(\langle ek_\mathbf{y} \rangle_0, \langle ek_\mathbf{y} \rangle_1)$ and distributes the pair $(\mathbf{I}_\mathbf{y}, \langle ek_\mathbf{y} \rangle_\sigma)$ to S_σ.
2. **Fuzzy Retrieval.** Servers navigate the hierarchical tree to locate the relevant leaf cluster:
   (a) *Initialization:* Set $\mathcal{C}_{curr} \leftarrow$ root clusters of $\{\mathcal{T}_t\}$.
   (b) *Iterative Selection:* Execute the following operations iteratively until $\mathcal{C}_{curr}$ becomes a leaf node:
      - For each $\mathcal{E}_k \in \mathcal{C}_{curr}$, servers compute distance shares $\langle dist_k \rangle_\sigma$ between $\mathbf{y}$ and $\mathbf{x}_k$ using `SEDS/SCDS`.
      - Servers execute `SM` to identify the cluster $\mathcal{E}_{k^*}$ with minimum distance.
      - Update $\mathcal{C}_{curr} \leftarrow$ children of $\mathcal{E}_{k^*}$.

(2*) **Exact Retrieval.** Within the leaf node $\mathcal{E}_{leaf}$:
   (a) *Distance Computation:* For each $i^* \in \mathcal{E}_{leaf}$, servers compute $\langle dist_{i^*} \rangle_\sigma$ between $\mathbf{y}$ and $\mathbf{f}_{i^*}$ using `SEDS/SCDS`.
   (b) *Top-K Selection:* Servers execute `SM` to identify the K indices with the smallest distances.

Output: For **Fuzzy Retrieval**, directly return the images corresponding to all feature vectors in $\mathcal{E}_{leaf}$ to QU; For **Exact Retrieval**, return the top-K images.

Figure 2: Privacy-preserving Multi-layer Clustering Image Retrieval using Vector-HSS

Setup: Public key $pk = (N, g, \{h_j\}_{j=1}^{c})$.
Initialization:
$\mathbf{P_A}$: Given an $r \times c$ matrix $\mathbf{W}$, sample batch parameters $(\{\psi_k\}_{k=1}^{l}, \{\phi_k\}_{k=1}^{l})$.

1. Decompose $\mathbf{W}$ into $\langle \mathbf{W} \rangle_0$ and $\langle \mathbf{W} \rangle_1$, where each component is uniformly sampled from $[0, 2^{|m|+1+\kappa}]^{(r \times c)}$, such that $\langle \mathbf{W} \rangle_0 - \langle \mathbf{W} \rangle_1 = \mathbf{W}$.
2. For each row vector in $\mathbf{W}$, compute
$$\{ask_i\}_{i=1}^{r} = \prod_{j=1}^{c} h_j^{\mathbf{W}[i][j]}.$$
3. Send $(\langle \mathbf{W} \rangle_\sigma, \{ask_i\}_{i=1}^{r}, (\{\psi_k\}_{k=1}^{l}, \{\phi_k\}_{k=1}^{l}))$, the share data must be securely transmitted to the corresponding server.

$\mathbf{P_B}$: Given an input $\mathbf{x} \in \mathbb{Z}_N^{c \times 1}$.

1. Sample a random value $ek_{\mathbf{x}} \leftarrow_R [N^2]$ and split it into a pair of evaluation keys $(\langle ek_{\mathbf{x}} \rangle_0, \langle ek_{\mathbf{x}} \rangle_1)$ such that $\langle ek_{\mathbf{x}} \rangle_1 - \langle ek_{\mathbf{x}} \rangle_0 = ek_{\mathbf{x}}$.
2. Compute a vector ciphertext $\mathbf{I_x}$ such that for each coordinate $j \in [c]$,
$$\mathbf{I_x}[j] = h_j^{ek_{\mathbf{x}}} \cdot (1+N)^{\mathbf{x}[j]}.$$
3. Send $(\mathbf{I_x}, \langle ek_{\mathbf{x}} \rangle_\sigma)$, the share data must be securely transmitted to the corresponding server.

Inference:
$\mathbf{S}_\sigma$: Upon receiving messages from P_A and P_B, perform the `MVM` protocol and output the result share $\langle \mathbf{Z} \rangle_\sigma$.

Figure 3: Secure Outsourced Two-party Inference Framework using Vector-HSS

two-party outsourced inference algorithm shown in Fig. 3.

Following Protocol 4, our inference algorithm enables collaborative computation of $\mathbf{Wx}$ without exposing the user's primary vector $\mathbf{x}$ or the model holder's matrix $\mathbf{W}$. The result is output as subtractive secret shares that satisfy $\mathbf{Z}_0 - \mathbf{Z}_1 = \mathbf{W} \cdot \mathbf{x}$. The complete workflow is illustrated in Fig. 3.

7 Performance Evaluation

In this section, we present a comprehensive performance evaluation of Vector-HSS through theoretical analysis and a series of experiments. In the theoretical analysis part, we compared our Vector-HSS scheme with existing HSS schemes, focusing on communication and computational complexity on vector operations (details are provided in Appendix D due to space limitations). The results show that when computing the vector inner product, the communication complexity of Vector-HSS is comparable to that of existing HSS schemes, while its computational complexity is significantly lower.

In the experimental evaluation part, we first compare the computational overhead of Vector-HSS with existing HSS schemes. Subsequently, we evaluate the practical performance of Vector-HSS using our privacy-preserving image retrieval algorithm and secure two-party inference framework. The detailed experimental evaluations are as follows.

7.1 Evaluation of Vector-HSS

All experiments are implemented in Python 3.12 and executed on a Windows 11 workstation equipped with an Intel Core i5-12600KF CPU (12 cores, 3.70 GHz) and 32 GB of RAM. To ensure statistical significance, we report the mean performance across 100 independent trials. For cryptographic primitives, we employ the GNU Multiple Precision Arithmetic Library (GMP) via the mpz_t data type to ensure high-precision computation and efficiency. In our experiments, we set p' and q' as 512-bit integers, resulting in $N = (2p'+1)(2q'+1)$. The experimental objective is to compare the time required for computing the inner products between one query vector and 20 database vectors. Each vector element is a random integer sampled from $\mathbb{Z}_{2^{32}}$. *It is worth noting that, to ensure a fair comparison, we focus on the core operation of Vector-HSS and existing HSS schemes: the inner product (IP) computation. Since existing HSS schemes [30, 38] require a* `ConvertInput` *operation to prepare data for multiplication, we include this overhead as part of their total IP computation cost.*

As shown in Table 2, we compare our Vector-HSS with the schemes in [30, 38] for different vector dimensions, focusing on the computational overhead of `ConvertInput` and IP. From Table 2, when calculating 20 inner products over 1024-dimensional vectors, the `ConvertInput` time in [30]

Table 2: Computational Overhead Comparison (Time: s).

Ope.	Dim.	[11]	[12]	Vector-HSS (No batch)	Vector-HSS (Batch)
Con.	128	19.5	2.8	–	–
	256	40.3	6.1	–	–
	512	76.9	11	–	–
	1024	152.6	21.8	–	–
IP	128	3.2	3	1.4	0.08
	256	6.8	6.4	2.6	0.16
	512	12.4	12	5.2	0.31
	1024	25	24.2	10.8	0.63

Notations. Ope. denotes the operation performed by the server. **Dim.** denotes the vector's dimension. **Con.** denotes the `ConvertInput` operation. **IP** denotes inner product computation.

is 21.8s, which is much less than the 152.6s in the scheme from [38]. This is because the scheme in [30] reduces the number of `ConvertInput` operations by compressing the key length. In contrast, our Vector-HSS avoids `ConvertInput`, further eliminating this computational overhead.

The time for computing IP in schemes [30, 38] is approximately 25s, whereas it is about 10.8s for non-batch mode and 0.63s in batch mode of our Vector-HSS (the size of batch is 20). This difference arises because these two schemes are required to compute $20 \times 2 \times 1024$ exponentiations when computing 20 IPs, while Vector-HSS requires only 20×1025 exponentiations, especially, only 1025 exponentiations are needed in batch mode.

Fig. 4 illustrates the total computation overhead of the inner product calculation stage in Vector-HSS and existing HSS schemes [30, 38]. For a dimension size of 1024, the schemes from [38] and [30] incur overheads of 175.6s and 46s, respectively; In contrast, Vector-HSS requires only 10.8s in non-batch mode and 0.63s in batch mode. Although the total computational overhead of all schemes increases linearly with the increment of the number of dimensions, our computational overhead increases more slowly, especially in batch mode.

Overall, the results show that Vector-HSS significantly outperforms existing schemes [30, 38] in all test dimensions, achieving a 280× improvement over the scheme [38] and a more than 70× improvement over the scheme [30]. This validates the scalability and applicability of Vector-HSS in high-dimensional vector computations.

7.2 Evaluation of Image Retrieval

In this subsection, we evaluate the performance of our image retrieval algorithm proposed in Fig. 2. Experimental images are chosen uniformly from the CIFAR-10 benchmark dataset, the ResNet50 model is used for feature extraction, and principal component analysis is applied to adjust the feature vector's dimension. To ensure security, we set parameters according to Eq. (2) in Theorem 4. With our configuration, from precision $deg = 10^5 \approx 2^{17}$, we have $|m| = 17$ and $d > 512/17 \approx 30$. Therefore, experiments in this part use $d \geq 32$ to satisfy this security condition.

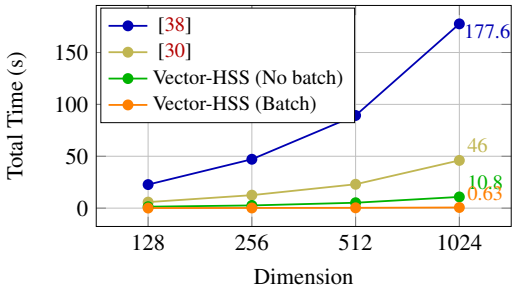


Figure 4: Total Time of Different HSS.

Note that we do not simulate real multi-party interactions. All participants, including users and servers, are instantiated as independent processes running on the same computer. This setup helps isolate and accurately measure the computational overhead of our retrieval scheme, eliminating the impact of factors such as network latency.

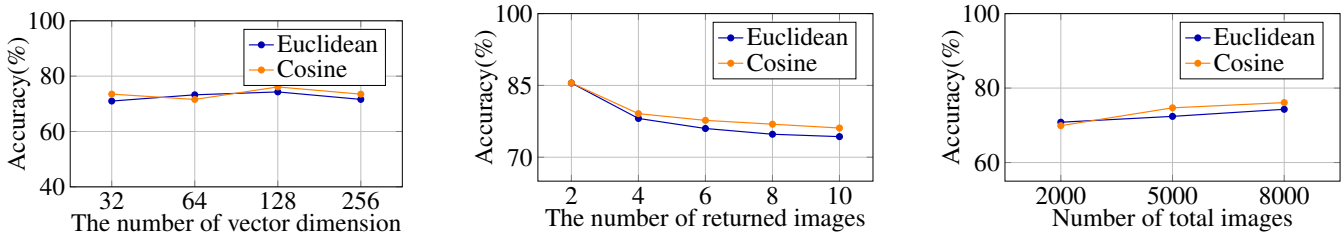
Accuracy Comparison Between Different Metrics. For a query vector $\mathbf{y}$, precision at K is defined as follows:

$$P@K = num_pos/K. \tag{6}$$

Here, *num_pos* denotes the number of relevant images correctly retrieved for the query vector $\mathbf{y}$, and K refers to the number of returned images. Fig. 5 shows the impact of different factors on retrieval accuracy in a precise retrieval scenario, including vector dimensionality d, the number of images returned K, and the total number of images $\mathcal{N}$.

Fig. 5(a) illustrates the impact of the number of vector dimensions on accuracy, under the condition of a fixed number of returned images ($K = 10$) and a fixed total image count ($\mathcal{N} = 8000$). It shows that the accuracy exhibits negligible variation with the change of the number of dimensions. Specifically, using Euclidean distance achieves an accuracy of about 72.5%, compared to about 73.5% with cosine distance. Fig. 5(b) demonstrates the effect of the number of returned images K on accuracy, with the vector dimension fixed at $d = 128$ and the total count of images at $\mathcal{N} = 8000$. As shown in Fig. 5(c), the accuracy consistently improves as the total number of images increases $\mathcal{N}$, with the vector dimension fixed at $d = 128$ and the number of returned images $K = 10$. As K increases, the accuracy shows a declining trend.

Accuracy Comparison Between Different Strategies. We compare the accuracy of *fuzzy* retrieval and *exact* retrieval, where fuzzy retrieval means returning a variable number of results (all items in the matched leaf cluster) and exact retrieval denotes returning an exactly fixed number K of results.



(a) $K = 10$ and $\mathcal{N} = 8000$ (b) $d = 128$ and $\mathcal{N} = 8000$ (c) $d = 128$ and $K = 10$

Figure 5: Accuracy of Our Image Retrieval under Different Metrics.

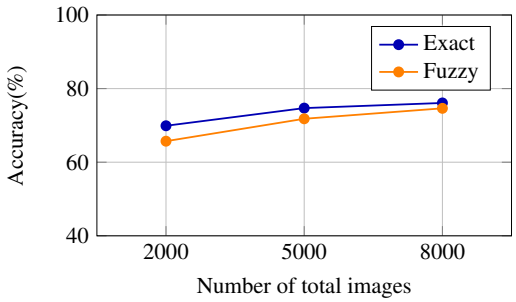


Figure 6: Accuracy of Different Strategies $d = 128$ and $K = 10$.

To minimize accuracy differences between fuzzy and exact retrieval, we kept the number of vectors per leaf node on fuzzy retrieval close to K.

From Fig. 6, under the cosine distance metric, exact retrieval consistently achieves higher accuracy than fuzzy retrieval. Notably, the difference is larger on small datasets than on bigger datasets. As the dataset size increases, the performance gap between the two strategies gradually narrows. This is likely because clustering on a limited dataset is more susceptible to noise, i.e., irrelevant images may be assigned to a target leaf node. On the other hand, fuzzy retrieval returns all vectors matched the leaf node, thus reducing accuracy. In contrast, exact retrieval mitigates this issue by selecting only the top-K nearest neighbors, effectively filtering out noisy candidates. As the dataset grows, clustering becomes more stable, and each leaf node contains sufficient relevant candidates, allowing the accuracy of fuzzy retrieval to gradually approach that of exact retrieval.

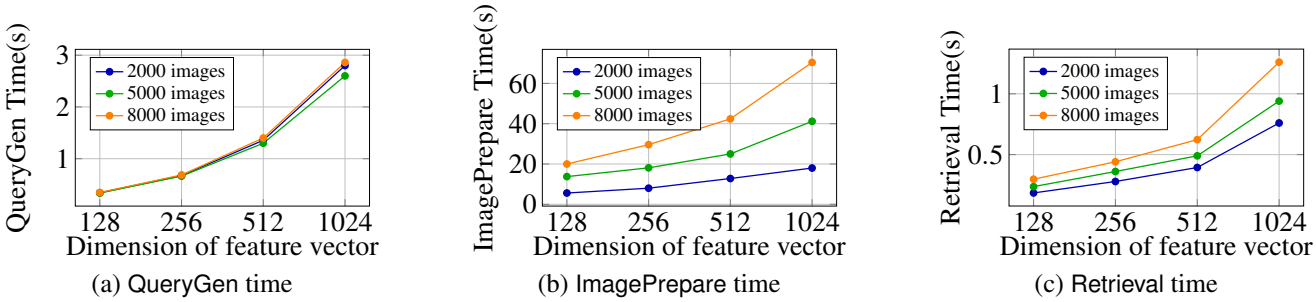
Efficiency Comparison Under Different $\mathcal{N}$. This experiment evaluates the computational overhead of our retrieval scheme under fuzzy retrieval across different dataset sizes: $\mathcal{N} = 2000, 5000$, and 8000. We adopt a three-layer clustering structure. To ensure that the number of returned images is approximately consistent across different dataset sizes, we adjust the number of clusters in each layer (i.e., the batch size in Vector-HSS) according to the dataset scale. Specifically, we use 5 clusters for 2000 images, 8 clusters for 5000 images, and 10 clusters for 8000 images.

Figure 7(a) illustrates QueryGen, which is the time cost for a query user to generate a query request. The results indicate that while the generation time increases with the number of vector dimensions, it remains independent of the dataset size. This is because query generation involves only the local processing of the user's query vector. Consequently, the performance of this step is determined solely by the number of vector dimensions. In contrast, as illustrated in Fig. 7(b) and 7(c), the overhead for image owners during the image preparation phase and the overhead for the query user during the image retrieval phase are both affected by the vector dimension and the size of the image dataset. It can be observed that as the vector dimension and dataset size increase, the execution time of these functions also increases.

Table 3: Comparison of Retrieval Efficiency (Time: s).

Scheme	Dim.	QueryGen	ImagePrepare	Retrieval
[6]	128	0.002	0.8	0.21
	512	0.009	12.6	2.68
	1024	0.047	78.3	20.48
[8]	128	0.3	28.9	460.00
	512	–	–	–
	1024	–	–	–
[9]	128	3.5	3.7	16.8
	512	13.8	14.7	36.8
	1024	29.8	31.5	68.6
Ours	128	0.3	5.6	0.19
	512	1.4	12.8	0.39
	1024	2.8	18	0.76

Efficiency Comparison with Prior Work ($\mathcal{N} = 2000$). As shown in Table 3, the retrieval time of the scheme in [45] is significantly higher than that of the other schemes. In particular, it fails to complete the retrieval operation within a reasonable time for 512-dimensional vectors and is therefore excluded from our tests in higher-dimensional scenarios. Among the remaining schemes, the retrieval time of the CKKS-based scheme [18] increases markedly with the vector dimension, reaching 68.6s at 1024 dimensions, which makes it unsuitable



(a) QueryGen time (b) ImagePrepare time (c) Retrieval time

Figure 7: Efficiency under Different Dataset Size and the Number of Vector Dimension.

for efficient retrieval. The secure KNN-based scheme [41] achieves sub-second retrieval efficiency in low-dimensional settings; however, its retrieval time also grows rapidly as the dimension increases, reaching 20.48s at 1024 dimensions. In contrast, our scheme maintains sub-second retrieval time across all dimensions, achieving only 0.76s even at 1024 dimensions, which significantly outperforms existing schemes and demonstrates better scalability and practicality in high-dimensional scenarios.

7.3 Evaluation of Two-Party Inference

In this subsection, the efficiency of the secure two-party inference framework in Section 6.2 is assessed, with a focus on the performance of the initialization and inference phases.

As shown in Table 4, during the initialization phase, the computational cost of P_A is significantly higher than that of P_B. This is because P_A handles complex matrix operations, while P_B only processes a vector. The computational workload and data size for P_A expand rapidly with the increment of the number of dimensions. Experimental results show that during the inference phase, the server (S_σ) achieves 0.56s inference time for a 128×128 matrix and maintains 8.47s computational efficiency even in higher dimensional scenarios such as 512×512.

Table 4: Efficiency of Our Inference Scheme (Time: s).

Step	**Party**	**Dimension** ($r \times c$)		
		128×128	256×256	512×512
Initialization	P_A	0.22	0.84	3.2
	P_B	0.002	0.006	0.01
Inference	S_σ	0.56	2.1	8.47

8 Related Work

Boyle et al. [12, 15] were the first to systematically formalize homomorphic secret sharing (HSS) and to propose DDH-based constructions that support distributed, non-interactive evaluation of branching programs. A key component of these schemes is share conversion, which non-interactively transforms shares from a multiplicative group to an additive group using a probabilistic DDLog technique. However, this conversion procedure is only probabilistically correct, incurring an inverse-polynomial error probability ε. Moreover, reducing this error is costly, as the computational overhead of homomorphic evaluation grows proportionally with $O(1/\varepsilon)$ [38]. While subsequent works [9, 14, 25] significantly improved the time–error trade-off of the probabilistic DDLog procedure, the resulting constructions still relied on probabilistic share conversion and therefore could not eliminate the error probability entirely.

Motivated by this limitation, researchers explored HSS constructions under alternative assumptions such as DCR [20, 26, 38, 40]. In particular, although Fazio et al. [26] proposed a Paillier-based HSS scheme, the scheme still relied on a probabilistic DDLog share conversion and thus incurred a non-negligible error probability. A major breakthrough came in 2021, when Orlandi et al. [38] introduced a new DDLog technique for the Paillier group. This technique achieves perfect correctness under standard number-theoretic assumptions and supports an exponential plaintext space, thereby establishing the foundation of modern DCR-based HSS.

While Orlandi et al.'s construction [38] resolves the fundamental correctness issues of prior HSS schemes, its implementation introduces a new efficiency bottleneck. Specifically, to avoid modular overflow in DDLog computations based on modulus N, the scheme employs a key decomposition technique, which leads to a significant increase in the number of `ConvertInput` operations invoked. This bottleneck motivated subsequent research to pursue improvements from different directions. On the computation side, Multi-Key HSS (MKHSS) [30] employs short-key technique to avoid key decomposition, thereby significantly reducing the number of `ConvertInput` calls. On the communication side, Abram et al. [1] achieve sublinear communication complexity by introducing Succinct-HSS, which decouple share size from linear dependence on the input length. However, for high-dimensional vectors, these schemes still require per-component `ConvertInput` calls. The number of such

calls grows linearly with the vector dimension, which limits the overall efficiency of HSS.

9 Conclusion

This paper introduces Vector-HSS, a high-performance homomorphic secret sharing scheme tailored for secure high-dimensional vector computation. By introducing a novel vector-centric computation paradigm, Vector-HSS eliminates the reliance on `ConvertInput` in traditional HSS schemes, thereby avoiding its repetitive overhead in high-dimensional settings. In addition, the scheme incorporates a batching technique based on CRT, which effectively reduces redundant exponentiations and further improves computational efficiency. To demonstrate its practical utility, we develop a suite of secure protocol modules and deploy them in two critical privacy-preserving tasks: image retrieval and bidirectional inference (PIR & PPML). Extensive experiments demonstrate that Vector-HSS significantly outperforms existing benchmarks, offering a scalable solution for privacy-preserving large-scale vector analytics.

References

[1] Damiano Abram, Lawrence Roy, and Peter Scholl. Succinct homomorphic secret sharing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 301–330, 2024.

[2] Nitin Agrawal, Ali Shahin Shamsabadi, Matt J Kusner, and Adrià Gascón. QUOTIENT: Two-party secure neural network training and prediction. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 1231–1247, 2019.

[3] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (h) ibe in the standard model. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 553–572, 2010.

[4] Shweta Agrawal, Benoît Libert, and Damien Stehlé. Fully secure functional encryption for inner products, from standard assumptions. In *Annual International Cryptology Conference*, pages 333–362, 2016.

[5] Shashank Balla and Farinaz Koushanfar. HELiKs: HE linear algebra kernels for secure inference. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2306–2320, 2023.

[6] James Bartusek, Sanjam Garg, Daniel Masny, and Pratyay Mukherjee. Reusable two-round mpc from ddh. In *Theory of Cryptography Conference*, pages 320–348, 2020.

[7] Dan Bogdanov, Sven Laur, and Jan Willemson. Sharemind: A framework for fast privacy-preserving computations. In *European Symposium on Research in Computer Security*, pages 192–206, 2008.

[8] Dan Bogdanov, Margus Niitsoo, Tomas Toft, and Jan Willemson. High-performance secure multi-party computation for data mining applications. *International Journal of Information Security*, 11(6):403–418, 2012.

[9] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: optimizations and applications. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2105–2122, 2017.

[10] Elette Boyle, Geoffroy Couteau, and Pierre Meyer. Sublinear-communication secure multiparty computation does not require fhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 159–189, 2023.

[11] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367, 2015.

[12] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under ddh. In *Annual International Cryptology Conference*, pages 509–539, 2016.

[13] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1292–1303, 2016.

[14] Elette Boyle, Niv Gilboa, and Yuval Ishai. Group-based secure computation: optimizing rounds, communication, and computation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 163–193, 2017.

[15] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. *Cryptology ePrint Archive*, 2017.

[16] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without fhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33, 2019.

[17] Zvika Brakerski and Shafi Goldwasser. Circular and leakage resilient public-key encryption under subgroup indistinguishability: (or: Quadratic residuosity strikes back). In *Annual Cryptology Conference*, pages 1–20, 2010.

[18] Lin Chen, Yiwei Yang, Li Yang, Chao Xu, Yinbin Miao, Zhiquan Liu, Ximeng Liu, Kim-Kwang Raymond Choo, Zhihong Liang, and Chao Hong. Efficient and secure content-based image retrieval in cloud-assisted internet of things. *IEEE Internet of Things Journal*, 12(5):6001–6013, 2024.

[19] Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel Paul Smart, and Barry Van Leeuwen. Scooby: improved multi-party homomorphic secret sharing based on fhe. In *International Conference on Security and Cryptography for Networks*, pages 540–563, 2022.

[20] Arka Rai Choudhuri, Aarushi Goel, Aditya Hegde, and Abhishek Jain. Homomorphic secret sharing with verifiable evaluation. In *Theory of Cryptography Conference*, pages 614–650, 2025.

[21] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. Single-server private information retrieval with sublinear amortized time. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33, 2022.

[22] Geoffroy Couteau, Lalita Devadas, Aditya Hegde, Abhishek Jain, and Sacha Servan-Schreiber. Multi-key homomorphic secret sharing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33, 2025.

[23] Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 194–224, 2023.

[24] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear mpc from sparse lpn. In *Annual International Cryptology Conference*, pages 315–348, 2023.

[25] Itai Dinur, Nathan Keller, and Ohad Klein. An optimal distributed discrete log protocol with applications to homomorphic secret sharing. In *Annual International Cryptology Conference*, pages 213–242, 2018.

[26] Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E Skeith III. Homomorphic secret sharing from paillier encryption. In *International conference on provable security*, pages 381–399, 2017.

[27] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 640–658, 2014.

[28] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive*, 2023.

[29] Jiaxing He, Kang Yang, Guofeng Tang, Zhangjie Huang, Li Lin, Changzheng Wei, Ying Yan, and Wei Wang. Rhombus: Fast homomorphic matrix-vector multiplication for secure two-party inference. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2490–2504, 2024.

[30] Kaiwen He, Sacha Servan-Schreiber, Geoffroy Couteau, and Srinivas Devadas. Concretely-efficient multi-key homomorphic secret sharing and applications. *Cryptology ePrint Archive*, 2025.

[31] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure Two-Party deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.

[32] T Janani and M Brindha. SESIM: An improved privacy preserving image retrieval protocol over encrypted cloud database. *IEEE Transactions on Multimedia*, 24:3794–3806, 2021.

[33] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[34] Russell WF Lai, Giulio Malavolta, and Dominique Schröder. Homomorphic secret sharing for low degree polynomials. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 279–309, 2018.

[35] Dong Li, Qingguo Lü, Xiaofeng Liao, Tao Xiang, Jiahui Wu, and Junqing Le. Avpmir: Adaptive verifiable privacy-preserving medical image retrieval. *IEEE Transactions on Dependable and Secure Computing*, 21(5):4637–4651, 2024.

[36] Rui Li, Alex X Liu, Ying Liu, Huanle Xu, and Huaqiang Yuan. Insecurity and hardness of nearest neighbor queries over encrypted data. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1614–1617, 2019.

[37] Xiumin Li, Mi Wen, Siying He, Rongxing Lu, and Liangliang Wang. A privacy-preserving federated learning scheme against poisoning attacks in smart grid. *IEEE Internet of Things Journal*, 11(9):16805–16816, 2024.

[38] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and

public-key silent ot. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 678–708, 2021.

[39] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*, pages 223–238, 1999.

[40] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from dcr and applications. In *Annual International Cryptology Conference*, pages 687–717, 2021.

[41] Lin Song, Yinbin Miao, Jian Weng, Kim-Kwang Raymond Choo, Ximeng Liu, and Robert H Deng. Privacy-preserving threshold-based image retrieval in cloud-assisted internet of things. *IEEE Internet of Things Journal*, 9(15):13598–13611, 2022.

[42] Qiuyun Tong, Yinbin Miao, Lei Chen, Jian Weng, Ximeng Liu, Kim-Kwang Raymond Choo, and Robert H Deng. Vfirm: Verifiable fine-grained encrypted image retrieval in multi-owner multi-user settings. *IEEE Transactions on Services Computing*, 15(6):3606–3619, 2021.

[43] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation*, pages 299–313, 2017.

[44] Haining Yang, Ye Su, Jing Qin, and Huaxiong Wang. Privacy-preserving outsourced inner product computation on encrypted database. *IEEE Transactions on Dependable and Secure Computing*, 19(2):1320–1337, 2020.

[45] Tengfei Yang, Jianfeng Ma, Yinbin Miao, Yue Wang, Ximeng Liu, Kim-Kwang Raymond Choo, and Bin Xiao. Mu-teir: Traceable encrypted image retrieval in the multi-user setting. *IEEE Transactions on Services Computing*, 16(2):1282–1295, 2022.

[46] Peng Zhang, Teng Huang, Xiaoqiang Sun, Wei Zhao, Hongwei Liu, Shangqi Lai, and Joseph K Liu. Privacy-preserving and outsourced multi-party k-means clustering based on multi-key fully homomorphic encryption. *IEEE Transactions on Dependable and Secure Computing*, 20(3):2348–2359, 2022.

[47] Bowen Zhao, Jiaming Yuan, Ximeng Liu, Yongdong Wu, Hwee Hwa Pang, and Robert H Deng. SOCI: A toolkit for secure outsourced computation on integers. *IEEE Transactions on Information Forensics and Security*, 17:3637–3648, 2022.

A Proof of Theorems in Section 4.3

We present the proofs of Theorems 1 - 4. To simplify the proof, the following lemmas and remark will be used.

Lemma 1 ([3]). *Assume $d > (r+1)\log q + \omega(\log r)$ with $q > 2$ prime. Let C be uniformly sampled from $\{-1,1\}^{d\times c}$ mod q, where $c = c(n)$ is a polynomial function of n. Define E and B as random matrices drawn uniformly from $\mathbb{Z}_q^{r\times d}$ and $\mathbb{Z}_q^{r\times c}$ respectively. Then for any vector $w \in \mathbb{Z}_q^d$, the joint distribution $(E, EC, C^{\mathsf{T}}w)$ is statistically indistinguishable from $(E, B, C^{\mathsf{T}}w)$.*

Lemma 2 ([8]). *For input $x \in [-\alpha,\alpha]$, if $r \leftarrow_R [0, 2^{\log\alpha+\kappa}]$, then $\mathbf{SD}(x+r, U) \leq 2^{-\kappa}$, where U is uniform over the sampling interval and $\mathbf{SD}(X,Y)$ denotes the statistical distance between distributions X and Y..*

Remark 1 ([4]). *Let $N = pq$ be a safe modulus, and $g = g'^{2N}$ mod N^2, where $g' \leftarrow \mathbb{Z}_{N^2}^*$. Then g generates the subgroup of $(2N)$-th residues in $\mathbb{Z}_{N^2}^*$. Define two distributions: $\mathcal{D}_0 := \{C_0 = g^r \bmod N^2 : r \leftarrow \{1,\ldots,\lfloor N/4\rfloor\}$, and $\mathcal{D}_1 := \{C_0 = z^2 \bmod N^2 : z = z_0^N \bmod N^2, z_0 \leftarrow \mathbb{Z}_N^*\}$. Then*

$$\Delta(\mathcal{D}_0, \mathcal{D}_1) \leq \mathrm{negl}(\kappa).$$

Lemma 3 ([38]). *Under the DCR assumption, for any efficient adversary $\mathcal{A}$, its advantage in successfully breaking the interactive vector game (as defined in [17]) is negligible.*

A.1 Proof of Theorem 1

$$\begin{aligned}
\frac{g_0}{g_1} &= \frac{D_0 \cdot H_0}{D_1 \cdot H_1} \\
&= \frac{ask_{\mathbf{y}}^{\langle ek_{\mathbf{x}}\rangle_0} \cdot \prod_{i=1}^d \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{y}[i]\rangle_0}}{ask_{\mathbf{y}}^{\langle ek_{\mathbf{x}}\rangle_1} \cdot \prod_{i=1}^d \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{y}[i]\rangle_1}} \\
&= \frac{g^{\langle ek_{\mathbf{x}}\rangle_0 \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i]} \cdot \prod_{i=1}^d [(1+N)^{\mathbf{x}[i]} g^{\mathbf{v}[i] ek_{\mathbf{x}}}]^{\langle \mathbf{y}[i]\rangle_0}}{g^{\langle ek_{\mathbf{x}}\rangle_1 \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i]} \cdot \prod_{i=1}^d [(1+N)^{\mathbf{x}[i]} g^{\mathbf{v}[i] ek_{\mathbf{x}}}]^{\langle \mathbf{y}[i]\rangle_1}} \bmod N^2 \\
&= \frac{(1+N)^{\mathbf{x}\cdot\langle \mathbf{y}\rangle_0} g^{ek_{\mathbf{x}} \sum_{i=1}^d \mathbf{v}[i]\langle \mathbf{y}[i]\rangle_0 + \langle ek_{\mathbf{x}}\rangle_0 \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i]}}{(1+N)^{\mathbf{x}\cdot\langle \mathbf{y}\rangle_1} g^{ek_{\mathbf{x}} \sum_{i=1}^d \mathbf{v}[i]\langle \mathbf{y}[i]\rangle_1 + \langle ek_{\mathbf{x}}\rangle_1 \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i]}} \bmod N^2 \\
&= (1+N)^{\mathbf{x}\cdot\mathbf{y}} g^{ek_{\mathbf{x}} \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i] - ek_{\mathbf{x}} \sum_{i=1}^d \mathbf{v}[i]\mathbf{y}[i]} \bmod N^2 \\
&= (1+N)^{\mathbf{x}\cdot\mathbf{y}} \bmod N^2
\end{aligned}$$

Figure 8: Correctness proof for a single vector.

Proof. **For IP operation:** S_0 and S_1 compute $g_0 \leftarrow D_0 \cdot H_0$ and $g_1 \leftarrow D_1 \cdot H_1$, respectively. Since $\langle \mathbf{y}[i]\rangle_0 - \langle \mathbf{y}[i]\rangle_1 = \mathbf{y}[i], i \in [1,d]$ and $\langle ek_{\mathbf{x}}\rangle_1 - \langle ek_{\mathbf{x}}\rangle_0 = ek_{\mathbf{x}}$, the derivation in Fig. 8 can be performed, where $\mathbf{I}_{\mathbf{x}}$ in Fig. 8 is the primary

input value and $ask_{\mathbf{y}}$ is the auxiliary information. Finally, we have $g_0 = g_1 \cdot (1+N)^{\mathbf{x}\cdot\mathbf{y}} \bmod N^2$.

Since $\mathbf{DDLog}(g)$ proposed in [38] can accurately convert divisive shares to subtractive shares, it is evident that S_0 and S_1 can respectively compute $z_0 \leftarrow \mathbf{DDLog}(g_0)$ and $z_1 \leftarrow \mathbf{DDLog}(g_1)$, obtaining results that satisfy $z_0 - z_1 = \mathbf{x}\cdot\mathbf{y} \bmod N$.

$$
\begin{aligned}
\frac{g_0}{g_1} &= \frac{D_0' \cdot H_0'}{D_1' \cdot H_1'} \\
&= \frac{ASK^{\langle ek_{\mathbf{x}}\rangle_0} \cdot \prod_{i=1}^{d} \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{Y}\rangle_0}}{ASK^{\langle ek_{\mathbf{x}}\rangle_1} \cdot \prod_{i=1}^{d} \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{Y}\rangle_1}} \\
&= \frac{g^{\langle ek_{\mathbf{x}}\rangle_0 \sum_{j=1}^{l} \phi_j \sum_{i=1}^{d} \mathbf{v}[i]\mathbf{y}_j[i]}}{g^{\langle ek_{\mathbf{x}}\rangle_1 \sum_{j=1}^{l} \phi_j \sum_{i=1}^{d} \mathbf{v}[i]\mathbf{y}_j[i]}} \cdot \\
&\quad \frac{\prod_{i=1}^{d} [(1+N)^{\mathbf{x}[i]} g^{\mathbf{v}[i] ek_{\mathbf{x}}}]^{\sum_{j=1}^{l} \phi_j \langle \mathbf{y}_j[i]\rangle_0}}{\prod_{i=1}^{d} [(1+N)^{\mathbf{x}[i]} g^{\mathbf{v}[i] ek_{\mathbf{x}}}]^{\sum_{j=1}^{l} \phi_j \langle \mathbf{y}_j[i]\rangle_1}} \bmod N^2 \\
&= \frac{(1+N)^{\sum_{j=1}^{l} \phi_j \mathbf{x}\cdot\langle \mathbf{y}_j\rangle_0}}{(1+N)^{\sum_{j=1}^{l} \phi_j \mathbf{x}\cdot\langle \mathbf{y}_j\rangle_1}} \cdot \\
&\quad \frac{g^{\langle ek_{\mathbf{x}}\rangle_0 \sum_{j=1}^{l} \phi_j \sum_{i=1}^{d} \mathbf{v}[i]\mathbf{y}_j[i] + ek_{\mathbf{x}} \sum_{i=1}^{d} \mathbf{v}[i] \sum_{j=1}^{l} \phi_j \langle \mathbf{y}_j[i]\rangle_0}}{g^{\langle ek_{\mathbf{x}}\rangle_1 \sum_{j=1}^{l} \phi_j \sum_{i=1}^{d} \mathbf{v}[i]\mathbf{y}_j[i] + ek_{\mathbf{x}} \sum_{i=1}^{d} \mathbf{v}[i] \sum_{j=1}^{l} \phi_j \langle \mathbf{y}_j[i]\rangle_1}} \bmod N^2 \\
&= (1+N)^{\sum_{j=1}^{l} \phi_j \mathbf{x}\cdot\mathbf{y}_j} \bmod N^2
\end{aligned}
$$

Figure 9: Correctness proof for a set of vectors.

For BIP operation: We change the values of $D_0 \cdot H_0$ and $D_1 \cdot H_1$ to $ASK^{\langle ek_{\mathbf{x}}\rangle_0} \cdot \prod_{i=1}^{d} \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{Y}\rangle_0}$ and $ASK^{\langle ek_{\mathbf{x}}\rangle_1} \cdot \prod_{i=1}^{d} \mathbf{I}_{\mathbf{x}}[i]^{\langle \mathbf{Y}\rangle_1}$, the derivation in Fig. 9 can be performed. Same as above, we can acquire a set of subtractive shares through $z_0' \leftarrow \mathbf{DDLog}(g_0) \bmod \psi_j$ and $z_1' \leftarrow \mathbf{DDLog}(g_1) \bmod \psi_j$, which satisfy $z_{j,0}' - z_{j,1}' = \mathbf{x}\cdot\mathbf{y}_j \bmod \psi_j$. □

A.2 Proof of Theorem 2 (Extension of Lemma 1)

Proof. By the Chinese Remainder Theorem, it suffices to show statistical indistinguishability modulo p and modulo q. Modulo p, since E_p is uniform over $\mathbb{Z}_p^{r\times d}$ and $C_p \in \{-1,1\}^{d\times c}$, the leftover hash lemma implies that, for any fixed $\mathbf{w}_p \in \mathbb{Z}_p^d$ and $d > \frac{(r+1)\max\{\log p, \log q\} + \omega(\log r)}{|m|}$,

$$(E_p, E_p C_p, C_p^\top \mathbf{w}_p) \approx_s (E_p, B_p, C_p^\top \mathbf{w}_p),$$

where B_p is uniform over $\mathbb{Z}_p^{r\times c}$. The same argument holds modulo q under $d > (r+1)\log q + \omega(\log r)$. Since the components modulo p and q are independent, combining them via CRT preserves negligible statistical distance, which yields the claim over $\mathbb{Z}_N$. □

A.3 Proof of Theorem 3

Proof. The interval $[0, T-1]$ contains $T = T_1 + T_2$ integers. Under modulo T_1:

- Residues $0, 1, \ldots, T_2 - 1$ appear **twice** (from $\{0, \ldots, T_2 - 1\}$ and $\{T_1, \ldots, T_1 + T_2 - 1\}$)
- Residues $T_2, T_2 + 1, \ldots, T_1 - 1$ appear **once** (from $\{T_2, \ldots, T_1 - 1\}$)

Thus the probability mass function is:

$$P[a] = \begin{cases} \frac{2}{T_1+T_2}, & \text{for } a = 0, 1, \ldots, T_2 - 1, \\ \frac{1}{T_1+T_2}, & \text{for } a = T_2, T_2+1, \ldots, T_1 - 1. \end{cases}$$

The total variation distance is:

$$
\begin{aligned}
\mathbf{SD}(P,U) &= \frac{1}{2} \sum_{a=0}^{T_1-1} \left| P[a] - \frac{1}{T_1} \right| \\
&= \frac{1}{2} \left[T_2 \left(\frac{2}{T_1+T_2} - \frac{1}{T_1} \right) + (T_1 - T_2) \left(\frac{1}{T_1} - \frac{1}{T_1+T_2} \right) \right] \\
&= \frac{T_2(T_1 - T_2)}{T_1(T_1+T_2)} \le \frac{T_2}{T_1}.
\end{aligned}
$$

When $T_1 \gg T_2$, we have $\mathbf{SD}(P,U) \approx \frac{T_2}{T_1} = O\left(\frac{p'+q'}{p'q'}\right)$. □

A.4 Proof of Theorem 4

Proof. To prove the theorem, we use a standard hybrid argument. We will define a simulator $\mathcal{S}_{S_\sigma}$ by successive modifications to the random variable *REAL*. Eventually, any two subsequent random variables will be computationally indistinguishable.

Hyb_0: This random variable is distributed exactly as *REAL* during the real execution process of the protocol.

Hyb_1: In this hybrid, we replace $ask_{\mathbf{x}} = g^{\sum_{i=1}^{d} v_i x_i} \bmod N^2$ with $h_0 = g^{v_0} \bmod N^2$, where $v_0 \leftarrow_R \{1, \ldots, \lfloor N/4 \rfloor\}$. By Theorem 3, the sequence $\{v_i \bmod p'q'\}_{i=0}^{d}$ derived from $\{h_i\}_{i=0}^{d}$ is statistically indistinguishable from uniform over $\mathbb{Z}_{p'q'}^{d+1}$. Therefore, when $d > \frac{\max\{\log_2 p', \log_2 q'\}}{|m|}$, Theorem 2 implies that the distribution in this hybrid is statistically indistinguishable from the previous one.

Hyb_2: In this hybrid, we replace $\{h_i\}_{i=0}^{d}$ with $\{\hat{h}_i^{2N} \bmod N^2\}_{i=0}^{d}$, where $\{\hat{h}_i\} \leftarrow_R \mathbb{Z}_N^{*(d+1)}$. According to Remark 1, this hybrid is indistinguishable from the previous one.

Hyb_3: In this hybrid, we replace $\{\hat{h}_i^{2N} \bmod N^2\}_{i=0}^{d}$ with $\{h_i'\}_{i=0}^{d}$, where $\{h_i'\} \leftarrow_R \mathbb{Z}_{N^2}^{*(d+1)}$. Definition 1 guarantees that this hybrid is indistinguishable from the previous one.

Hyb_4: In this hybrid, we replace all the elements in $\langle \mathbf{x}[i]\rangle_\sigma$ with $\langle \mathbf{x'}[i]\rangle_\sigma \in [0, 2^{|m|+1+\kappa}]$ where $i \in [1, d]$. According to Lemma 2, this transformation preserves the distribution, making it statistically identical to the previous hybrid.

Hyb_5: In this hybrid, we replace the set $\mathbf{I}_\mathbf{y} = \{(1 + \mathbf{y}[i]N) \cdot h_i^{ek_\mathbf{y}} \bmod N^2\}_{i=1}^d$ with $\{h_i^{ek_\mathbf{y}} \bmod N^2\}_{i=1}^d$. According to Lemma 3, this hybrid is indistinguishable from the previous one.

Hyb_6: In this hybrid, we replace the set $\{h_i^{ek_\mathbf{y}} \bmod N^2\}_{i=1}^d$ with $\{(\hat{h}_i^{2N})^{ek_\mathbf{y}} \bmod N^2\}_{i=1}^d$. Identical to the analysis of Hyb_2, this hybrid is indistinguishable from the previous one.

Hyb_7: In this hybrid, we substitute the set $\{(\hat{h}_i^{2N})^{ek_\mathbf{y}} \bmod N^2\}_{i=1}^d$ with a uniformly random set $\{h_i''\}_{i=1}^d$, where each h_i'' is randomly sampled from $Z_{N^2}^*$. According to Definition 1, this hybrid is computationally indistinguishable from the previous one.

Hyb_8: In this hybrid, we replace $\langle ek_\mathbf{y} \rangle_\sigma$ with $\langle ek' \rangle_\sigma \in [0, 2^L]$, where $L = |ek_\mathbf{y}| + 1 + \kappa$. Identical to the analysis of Hyb_4, this hybrid is identically distributed to the previous hybrid.

Hyb_9: In this hybrid, we replace z_σ with $z'_\sigma = \mathbf{DDLog}(h_0'^{\langle ek' \rangle_\sigma} \cdot \prod_{i=1}^d h_i''^{\langle \mathbf{x}'[i] \rangle_\sigma})$. Since all elements used to compute z_σ have already been substituted with random values in previous hybrids, this hybrid is computationally indistinguishable from the previous one.

Therefore, we can construct a *PPT* simulator *SIM* that samples from the distribution characterized in the final hybrid. As shown by the above reasoning, the output of this simulator is computationally indistinguishable from that of *REAL*, which completes the proof. □

B Correctness Analysis of the Protocol Modules in Section 5

Theorem 5. *Through Protocol 1 and Protocol 2, the shares $\langle dist \rangle_0$ and $\langle dist \rangle_1$ can be correctly computed, and these two shares can be used to compare the numerical magnitudes of the squared Euclidean distances (or cosine distances) between $\mathbf{y}$ and $\mathbf{x}_1$, and between $\mathbf{y}$ and $\mathbf{x}_2$, respectively.*

Proof. **For the squared Euclidean distance:** Given that the squared Euclidean distance from vector $\mathbf{y}$ to $\mathbf{x}_1$ is $\sum_{i=1}^d (\mathbf{x}_1[i] - \mathbf{y}[i])^2$ and to $\mathbf{x}_2$ is $\sum_{i=1}^d (\mathbf{x}_2[i] - \mathbf{y}[i])^2$, their difference $\sum_{i=1}^d (\mathbf{x}_1[i] - \mathbf{y}[i])^2 - \sum_{i=1}^d (\mathbf{x}_2[i] - \mathbf{y}[i])^2 = \sum_{i=1}^d (\mathbf{x}_1[i]^2 - 2\mathbf{x}_1[i]\mathbf{y}[i] - \mathbf{x}_2[i]^2 + 2\mathbf{x}_2[i]\mathbf{y}[i])$ directly indicates their relative magnitudes. After computing the distance shares $\langle dist_1 \rangle_\sigma$ and $\langle dist_1 \rangle_\sigma$ using Protocol 1, the difference $\langle dist_1 \rangle_0 - \langle dist_2 \rangle_0 - (\langle dist_1 \rangle_1 - \langle dist_2 \rangle_1) = \sum_{i=1}^d (\mathbf{x}_1[i]^2 - 2\mathbf{x}_1[i]\mathbf{y}[i] - \mathbf{x}_2[i]^2 + 2\mathbf{x}_2[i]\mathbf{y}[i])$ is consistent with the previously computed result. Therefore, Protocol 1 can accurately compute the shares to compare the magnitudes of the squared Euclidean distances.

For the cosine distance: Similarly, by replacing the squared Euclidean distance with the cosine distance, we can obtain a new difference $(1 - \frac{\mathbf{x}_1 \cdot \mathbf{y}}{\|\mathbf{x}_1\| \|\mathbf{y}\|}) - (1 - \frac{\mathbf{x}_2 \cdot \mathbf{y}}{\|\mathbf{x}_2\| \|\mathbf{y}\|}) = \frac{\mathbf{x}_2 \cdot \mathbf{y}}{\|\mathbf{x}_2\| \|\mathbf{y}\|} - \frac{\mathbf{x}_1 \cdot \mathbf{y}}{\|\mathbf{x}_1\| \|\mathbf{y}\|}$. Through Protocol 1, new distance shares $\langle dist_1 \rangle_\sigma$ and $\langle dist_1 \rangle_\sigma$ can be computed, and the difference $\langle dist_1 \rangle_0 - \langle dist_2 \rangle_0 - (\langle dist_1 \rangle_1 - \langle dist_2 \rangle_1) = \frac{\mathbf{x}_2 \cdot \mathbf{y}}{\|\mathbf{x}_2\| \|\mathbf{y}\|} - \frac{\mathbf{x}_1 \cdot \mathbf{y}}{\|\mathbf{x}_1\| \|\mathbf{y}\|}$ remains consistent with the previously calculated result. Therefore, Protocol 2 can accurately compute the shares to compare the magnitudes of the cosine distances. □

Theorem 6. *The secure comparison (SC) protocol in Protocol 4 can correctly compare the magnitude relationship between two values x and y.*

Proof. For $dist_1, dist_2 \in [-2^{|m|}, 2^{|m|}]$, we observe that both $dist_1 - dist_2 + 1$ and $dist_2 - dist_1$ are bounded within $[-2^{|m|+1}, 2^{|m|+1} + 1]$. The protocol utilizes random values $r_1 \leftarrow \{0,1\}^\kappa \setminus \{0\}$ and $r_2 \leq N_P/2$ with $r_1 + r_2 > N_P/2$, ensuring that the core comparison expressions satisfy $r_1(dist_1 - dist_2 + 1) + r_2, r_1(dist_2 - dist_1) + r_2 \in \left[\frac{N_P}{2} - (2^\kappa + 2^{|m|+1+\kappa}), \frac{N_P}{2} + 2^{|m|+1+\kappa}\right]$. The significant size difference ($N_P/2 \gg 2^{|m|+1+\kappa}$) ensures that all values remain properly bounded within $(0, N_P)$.

The comparison logic emerges naturally from these bounds. When $0 < r_1(dist_1 - dist_2 + 1) + r_2 \leq N_P/2$, this corresponds to $dist_1 - dist_2 + 1 \leq 0$ and thus $dist_1 < dist_2$, causing the output 1. In contrast, values above $N_P/2$ indicate $dist_1 - dist_2 + 1 \geq 1$ and thus $dist_1 \geq dist_2$, resulting in 0. Similarly, $0 < r_1(dist_2 - dist_1) + r_2 \leq N_P/2$ implies $dist_2 - dist_1 \leq 0$ (or $dist_1 \geq dist_2$) with output 0, while the values above $N_P/2$ correspond to $dist_2 - dist_1 \geq 1$ (or $dist_1 < dist_2$) outputting 1. □

C Security Analysis of the Protocol Modules in Section 5

In this section, we present the security analysis of the four protocol modules described in this paper. To simplify the proofs, the following lemmas and remark will be used.

Lemma 4 ([7]). *If all subprotocols can be simulated, then the entire protocol can be perfectly simulated.*

Remark 2 ([37]). *The compression of a high-dimensional feature matrix into an integer prevents a curious server from recovering the original features.*

Theorem 7. *In the case of non-colluding adversaries $\mathcal{A}_{S_0}$ and $\mathcal{A}_{S_1}$, Protocol 1, Protocol 2, and Protocol 4 in Section 5 can all securely compute their respective result shares.*

Proof. These three protocols are constructed based on the BIP operation. According to the conclusion of Theorem 4, BIP can securely compute the shares $\{\langle \mathbf{x}_i \cdot \mathbf{y} \rangle_\sigma\}_{i=1}^l$ from the inputs $\{\langle \mathbf{x}_i \rangle_\sigma\}_{i=1}^l$ and $\mathbf{I}_\mathbf{y}$. It should also be noted that in protocol $\mathtt{SEDS}$, although the two servers need to negotiate to obtain the sum of squares of the cluster center vectors, Remark 2 has proven that knowing only this sum of squares does not reveal any information about the original cluster-center vectors. In summary, these three protocols can all securely generate their respective result shares according to Lemma 4. □

Table 5: Complexity Comparison with Existing HSS schemes

Scheme	Computational Complexity				Communication Complexity
	Data Preparation	ConvertInput	IP	BIP	
[38]	$O(2d\log^3 N)$	$O(d(\ell+1)\log^3 N)$	$O(2d\log^3 N)$	$O(2nd\log^3 N)$	$O(4d\log N)$
[30]	$O(d\log^3 N)$	$O(d\log^3 N)$	$O(2d\log^3 N)$	$O(2nd\log^3 N)$	$O(2d\log N)$
Vector-HSS	$O(d\log^3 N)$	None	$O(d\log^3 N)$	$O(\lceil \frac{n}{l} \rceil d\log^3 N)$	$O((d+1)\log N)$

Notations. n denotes the number of vectors, d represents the vector dimension, ℓ indicates the number of digits resulting from the key decomposition, and b defines the batch size.

Theorem 8. *In the case of non-collusive adversaries $\mathcal{A}_{S_0}$ and $\mathcal{A}_{S_1}$, the secure comparison(SC) protocol in Protocol 3 can securely compare the magnitude relationship between two values x and y.*

Proof. To simulate $\mathcal{S}_0$ and $\mathcal{S}_1$, we construct two independent simulators $\mathcal{S}_{S_0}$ and $\mathcal{S}_{S_1}$, respectively.

$\mathcal{S}_{S_0}$ simulates the view of $\mathcal{A}_{S_0}$ as follows.

First, $\mathcal{S}_{S_0}$ takes as input $\langle dist_x\rangle_0$, $\langle dist_y\rangle_0$, ζ and $res = \pi \oplus \mu$. Next, $\mathcal{S}_{S_0}$ randomly selects $\hat{x}$, $\hat{y}$, and $\hat{\pi} \leftarrow \{0,1\}$. If $\hat{\pi} = 0$, $\mathcal{S}_{S_0}$ computes $[\![\hat{\xi}]\!] \leftarrow [\![\hat{x} - \hat{y}] + 1]\!]$, if $\hat{\pi} = 1$, it computes $[\![\hat{\xi}]\!] \leftarrow [\![\hat{y} - \hat{x}]\!]$.

Compute $\hat{\mu}$ in reverse: $\hat{\mu} = res \oplus \hat{\pi}$, if $\hat{\mu} = 0$, $\mathcal{S}_{S_0}$ randomly selects $\hat{\zeta} \in \left(\frac{N_P}{2}, \frac{N_P}{2} + 2^{|m|+1+\kappa}\right]$ and calls Enc to encrypt -1 in $[\![-1]\!]$. Otherwise ($\hat{\mu} = 1$), $\mathcal{S}_{S_0}$ randomly selects $\hat{\zeta} \in \left(\frac{N_P}{2} - 2^{\kappa} - 2^{|m|+1+\kappa}, \frac{N_P}{2}\right]$ and proceeds to encrypt 1, producing the corresponding ciphertext $[\![1]\!]$.

Finally, $\mathcal{S}_{S_0}$ outputs the simulation of the view of $\mathcal{A}_{S_0}$, which includes $[\![\hat{\xi}]\!]$, $\hat{\zeta}$, and $[\![\hat{\pm 1}]\!]$.

Since the one-time key encryption scheme $x+u$ is proven to be chosen-plaintext attack secure by [8], $\hat{\zeta}$ is computationally indistinguishable from $\zeta = r_1 \cdot (x - y + 1) + r_2$ and $\zeta = r_1 \cdot (y - x) + r_2$. Furthermore, since $\hat{x}$, $\hat{y}$, $\hat{\pi}$, and $\hat{\pm 1}$ are randomly selected, and Enc is semantically secure, the view of $\mathcal{S}_{S_0}$ from the ideal model and the view of $\mathcal{A}_{S_0}$ from the real model are computationally indistinguishable.

$\mathcal{S}_{S_1}$ simulates the view of $\mathcal{A}_{S_1}$ as follows.

First, $\mathcal{S}_{S_1}$ takes as input $\langle dist_x\rangle_1$, $\langle dist_y\rangle_1$, $[\![\xi]\!]$, and $[\![\pm 1]\!]$. Next, $\mathcal{S}_{S_0}$ randomly selects $\widetilde{x}$, $\widetilde{y}$, $\widetilde{r_1} \leftarrow \{0,1\}^{\kappa} \setminus \{0\}$, and $\widetilde{r_2} \in \left(\frac{N_P}{2} - r_1, \frac{N}{2}\right]$.

$\mathcal{S}_{S_1}$ generates random $[\![\widetilde{\xi}]\!]$ and $[\![\widetilde{\pm 1}]\!]$(independent of π), then computes $[\![\widetilde{\zeta}]\!] = [\![\widetilde{r_1}(\widetilde{x} - \widetilde{y}) + \widetilde{r_2}]\!]$.

Finally, $\mathcal{S}_{S_1}$ outputs the simulation of the view of $\mathcal{A}_{S_1}$, which includes $[\![\widetilde{\xi}]\!]$, $[\![\widetilde{\zeta}]\!]$, and $[\![\widetilde{\pm 1}]\!]$.

Since $\widetilde{x}$, $\widetilde{y}$, $\widetilde{r_1}$, and $\widetilde{r_2}$ are randomly selected and Enc is semantically secure, the view of $\mathcal{S}_{S_1}$ from the ideal model and the view of $\mathcal{A}_{S_1}$ from the real model are computationally indistinguishable. □

D Theoretical Analysis of Performance

We consider the d-dimensional vectors settings and present a detailed comparison between our scheme and those schemes in [29, 38]. Regarding communication overhead, we compare the total communication volume of each scheme throughout the entire protocol flow. To facilitate a fair comparison of the computational overhead of different schemes under different frameworks, we divided the entire process into three stages based on the operational objectives: data preparation (it processes the raw data to prepare for subsequent inner product calculation), conversion execution, and (batched) inner product calculations. We conducted evaluations of the computational overhead of each stage to more accurately reflect the performance differences of each scheme in key operations. The results are summarized in Table 5. Note that, to ensure a fair comparison and to focus our evaluation on operations related to vector inner products, we exclude the communication and computational overhead of the key agreement phase in [29].

First, in the data preparation phase, the scheme in [38] requires computing a pair of input values (I_x, I'_x) for each element in the vector (detailed in Section 2). Since this involves computing two ciphertext elements, its computational complexity is $O(2d\log^3 N)$ when computing a d-dimensional vector inner product. In contrast, the scheme in [29] only needs to generate I'_x, thereby reducing the computational complexity to $O(d\log^3 N)$. Compared to both, the primary latency in our scheme also stems from computing the primary input value $\mathbf{I_x}$, resulting in the same computational complexity of $O(d\log^3 N)$.

During the `ConvertInput` phase, both schemes in [38] and [29] require executing the `ConvertInput` operation to transform ciphertexts into in-memory values for subsequent multiplication calculations. Specifically, in the scheme of [38], generating the share $\langle x\rangle$ requires only one conversion. However, generating $\langle xv\rangle$ involves key decomposition, which requires a conversion for each of the ℓ digits (details in Section 2), resulting in a cost as high as $O(d(\ell+1)\log^3 N)$. The scheme in [29] avoids key decomposition by shortening the key length, and only needs to generate $\langle xv\rangle$ via `ConvertInput`, from which the share $\langle y\rangle$ can be directly derived, thereby reducing its time overhead to $O(d\log^3 N)$. In comparison, our scheme eliminates the need for the `ConvertInput` operation and thus

incurs no corresponding overhead.

During the inner product computation phase, the schemes in [38] and [29] both require performing $2d$ exponentiations and modular multiplications, which corresponds to a computational complexity of $O(2d\log^3 N)$. In comparison, our scheme reduces the number of such operations to d, resulting in a complexity of $O(d\log^3 N)$. Furthermore, when computing inner products for n vectors, the schemes in [38] and [29] exhibit a computational complexity of $O(2nd\log^3 N)$ since they lack batch processing mechanisms. In contrast, by introducing a batch processing mechanism, our scheme reduces the complexity to $(\lceil \frac{n}{l} \rceil d\log^3 N)$.

Finally, regarding communication complexity, the scheme in [38] requires the transmission of (I_x, I'_x), which consists of four elements in $\mathbb{Z}^*_{N^2}$, resulting in the communication overhead $O(4d\log N)$. The scheme in [29] transmits only I'_x, containing two elements in $\mathbb{Z}^*_{N^2}$, which reduces the communication overhead to $O(2d\log N)$. In contrast, our scheme's primary transmission cost stems from the primary input value $\mathbf{I_x}$ and auxiliary information ask, further optimizing the communication overhead to $O((d+1)\log N)$. Note that, unlike the schemes [29, 38], our protocol depends on a secure channel to deliver shares of vector $\langle \mathbf{x} \rangle_\sigma$ and evaluation key $\langle ek_\mathbf{x} \rangle_\sigma$. Although this introduces stronger deployment assumptions, secure channels are achievable in many real-world two-party computation scenarios and can significantly improve protocol efficiency.